INTERACCIÓN

1. Introducción

2. Posicionamiento

3. Selección

4. Animación

"Nada es particularmente difícil si lo divides en pequeños trabajos "

Henry Ford

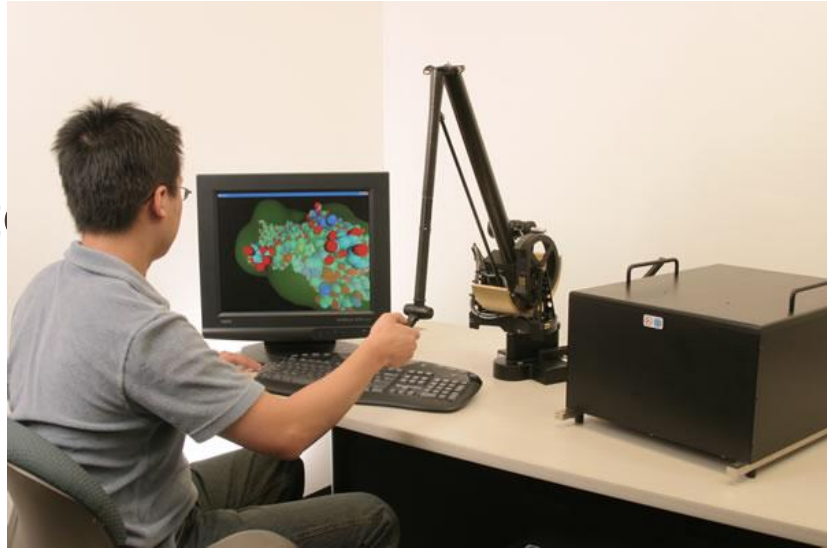
1. Introducción

- Un sistema gráfico interactivo es un sistema que responde de forma directa a las acciones del usuario.

- El sistema gráfico funciona según el siguiente ciclo:
  - El usuario realiza una acción
  - El programa recibe la acción
  - Procesa la acción
  - Redibuja el modelo

1. Introducción

- La incorporación de interactividad permite realizar aplicaciones que respondan a las acciones de los usuarios.
- En un sistema gráfico interactivo básico el usuario debe disponer de al menos un dispositivo de entrada y un dispositivo de visualización (mínimo teclado y monitor). Normalmente se dispone un ratón.
- En sistemas más profesionales se suelen utilizar dispositivos de entrada especiales como:
  - Tableta digitalizadora
  - Sistemas de posicionamiento
  - Palpadores

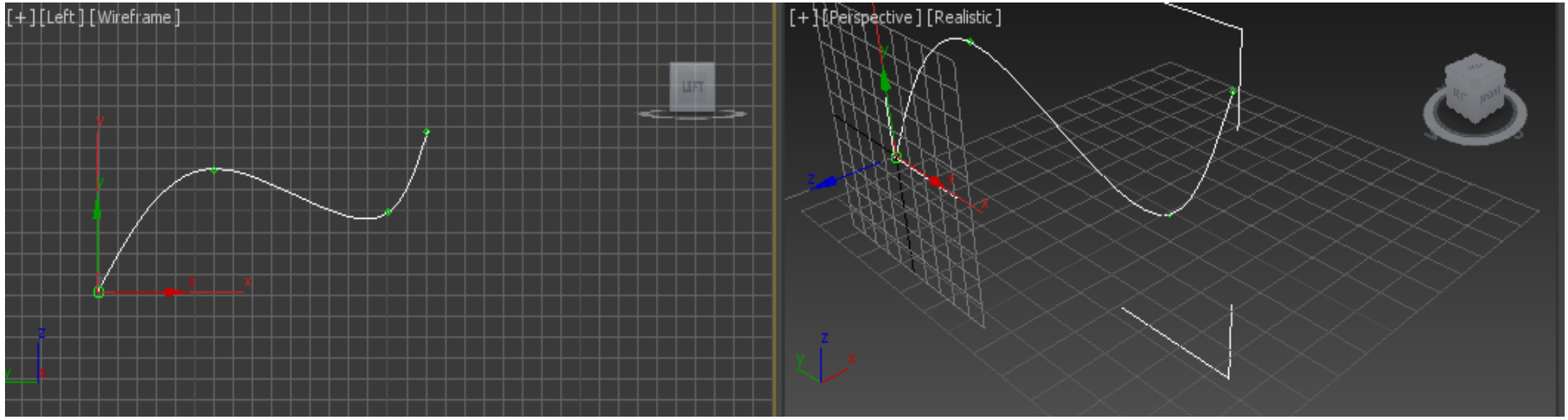


1. Introducción

- Latencia: para un sistema gráfico interactivo, la latencia (retardo entre la acción del usuario y la respuesta de un sistema) ha de ser lo suficientemente pequeña para que el usuario perciba una reacción de causa efecto.
  - Del orden de 100 milisegundos
- Realimentación: es el mecanismo mediante el cual el sistema da información al usuario útil para determinar la siguiente acción a realizar. Se puede usar con diferentes fines:
  - mostrar el estado del sistema
  - como parte de una función de entrada
  - para reducir la incertidumbre del usuario

1. Introducción (técnicas de entrada)

- Técnicas que simplifican el proceso de interacción:
  - Líneas o curvas elásticas
  - Rejillas
  - Puntos de gravedad



1. Introducción (dispositivos de entrada)

- El usuario introduce la información por medio de dispositivos de entrada.
- Estos pueden ser de propósito general (teclado), o específicos para la generación de datos geométricos (digitalizador).
- Podemos clasificar los dispositivos de entrada gráfica atendiendo a la información que generan. Esta puede ser:
  - Desplazamientos 2D: ratón, trackball, joystick
  - Posiciones 2D: tableta digitalizadora, lápiz óptico, pantalla táctil

1. Introducción

- Dispositivos de entrada (continuación)
  - Desplazamientos en 3D: spaceMouse.
  - Posiciones en 3D: brazos de medición, los dispositivos de seguimiento (trackers) y los hápticos.
  - Gestos: Kinect y Leap Motion

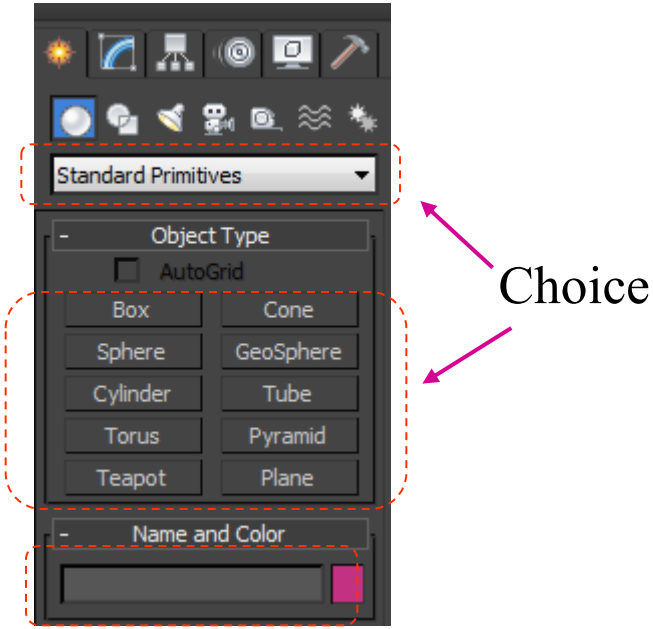
1. Introducción (dispositivos de entrada lógicos)

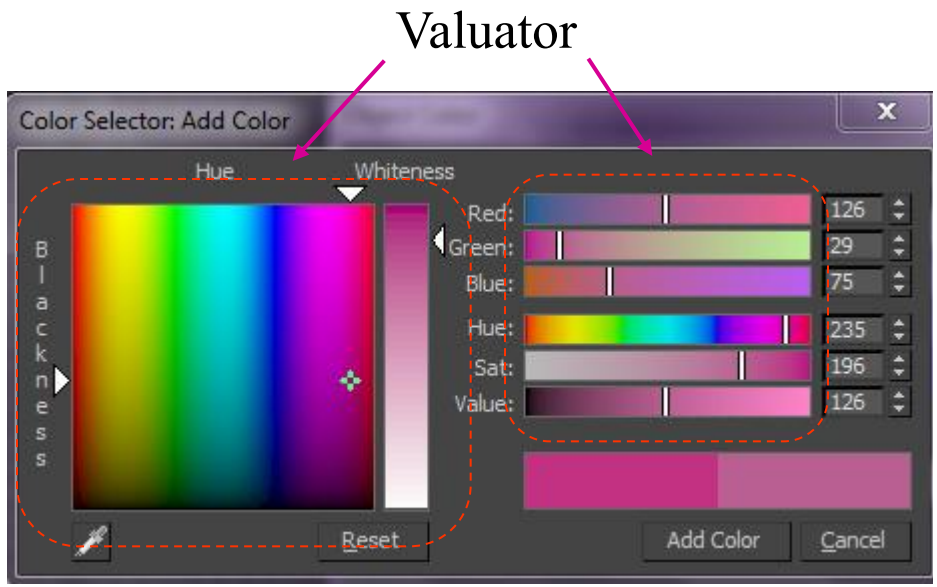
- En un sistema gráfico se utilizan dispositivos de entrada abstractos, denominados dispositivos de entrada lógicos.

- Cada dispositivo lógico representa un dispositivo físico ideal, que genera un único tipo de información de entrada.

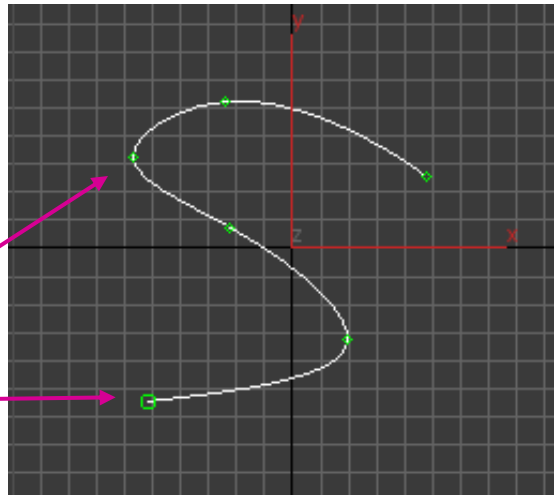
- Hay cinco tipos de dispositivos de entrada lógicos:
  - Locator (posicionamiento): un punto en coordenadas mundiales
  - Valuator: un valor numérico dentro de algún rango
  - Choice: selección de un valor dentro de un conjunto de alternativas
  - String: una cadena de texto
  - Pick (selección): un identificador de un componente de un modelo gráfico

1. Introducción (dispositivos de entrada lógicos)

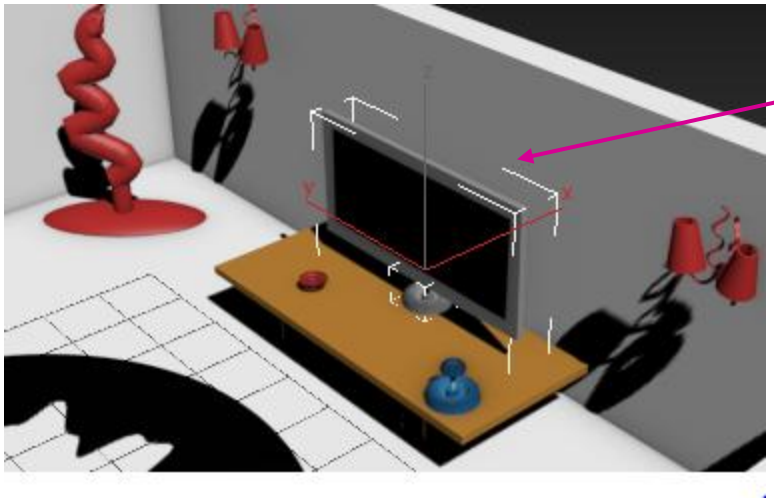




String



Locator



Pick

1. Introducción (dispositivos de entrada lógicos)

- Un dispositivo de entrada físico se puede corresponder con uno o varios dispositivos de entrada lógicos.
- Los dispositivos de entrada lógicos, en principio, se pueden hacer corresponder con cualquier dispositivo físico de entrada, pero algunos dispositivos hardware son más apropiados que otros para ciertos tipos de datos de entrada
- Para un sistema gráfico hay dos tipos de dispositivos lógicos más más específicos: Locator (posicionamiento) y Pick (selección).
  - El posicionamiento permite generar posiciones en coordenadas del mundo.
  - La selección permite identificar interactivamente elementos del modelo geométrico.

1. Introducción (Procesamiento)

- La forma en como interactúan los dispositivos de entrada y una aplicación depende de cómo se sincronicen las acciones del usuario con la aplicación.
- La sincronización debe realizarse de forma distinta dependiendo de la función a realizar.
- La sincronización se resuelve en los sistemas gráficos, haciendo abstracción de las operaciones de entradas
- Se definen tres modos de entrada: petición (request), evento (event) y muestreo (sample)
- En cada momento, la aplicación puede asociar un modo a cada dispositivo lógico.

1. Introducción (Procesamiento)

- El modo petición (request) funciona como una lectura sin buffer de entrada.

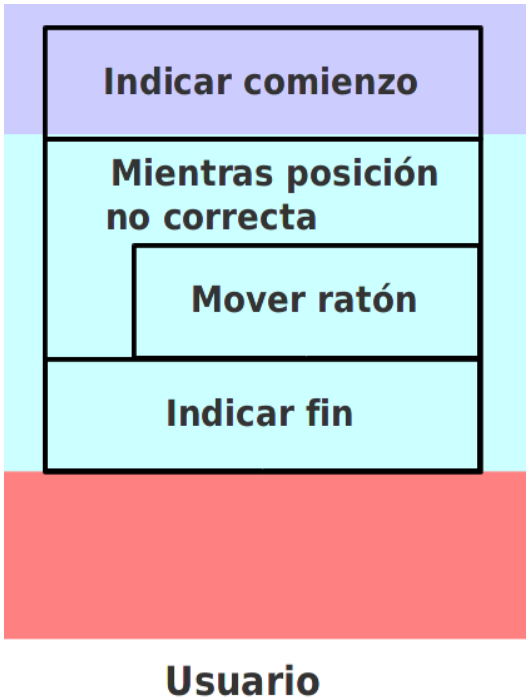
- En el modo evento (event) se utiliza un buffer, de modo que el usuario puede realizar entradas antes de que la aplicación las solicite. Se almacena una cola de sucesos(valores y dispositivo que lo produjo

- El modo muestreo (sample) se asocia un valor al dispositivo por defecto al dispositivo que el usuario puede modificar y la palicación puede consultar su valor.

2. Posicionamiento

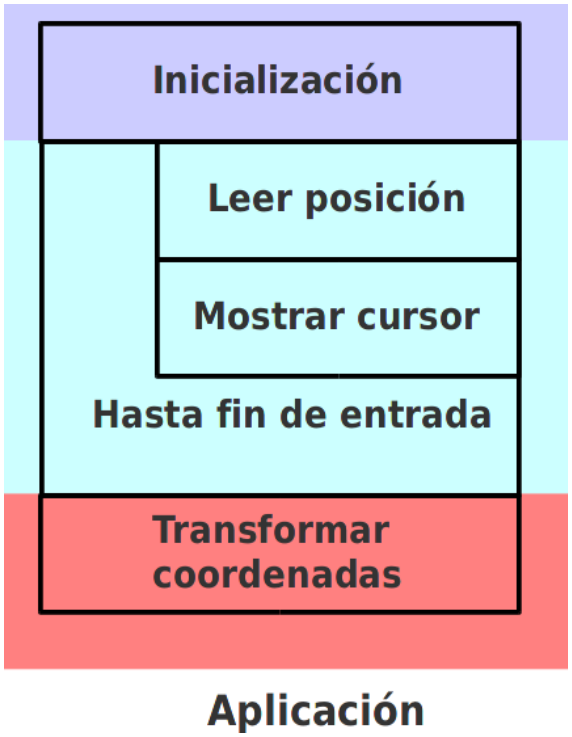
- El objetivo de las operaciones de posicionamiento es permitir la introducción de un punto.

- Desde el punto de vista del usuario, que utiliza un dispositivo que controla un cursor (dispositivo lógico), el proceso de entrada es el que siguiente
  - Indicar comienzo
  - Mover dispositivo hasta indicar fin



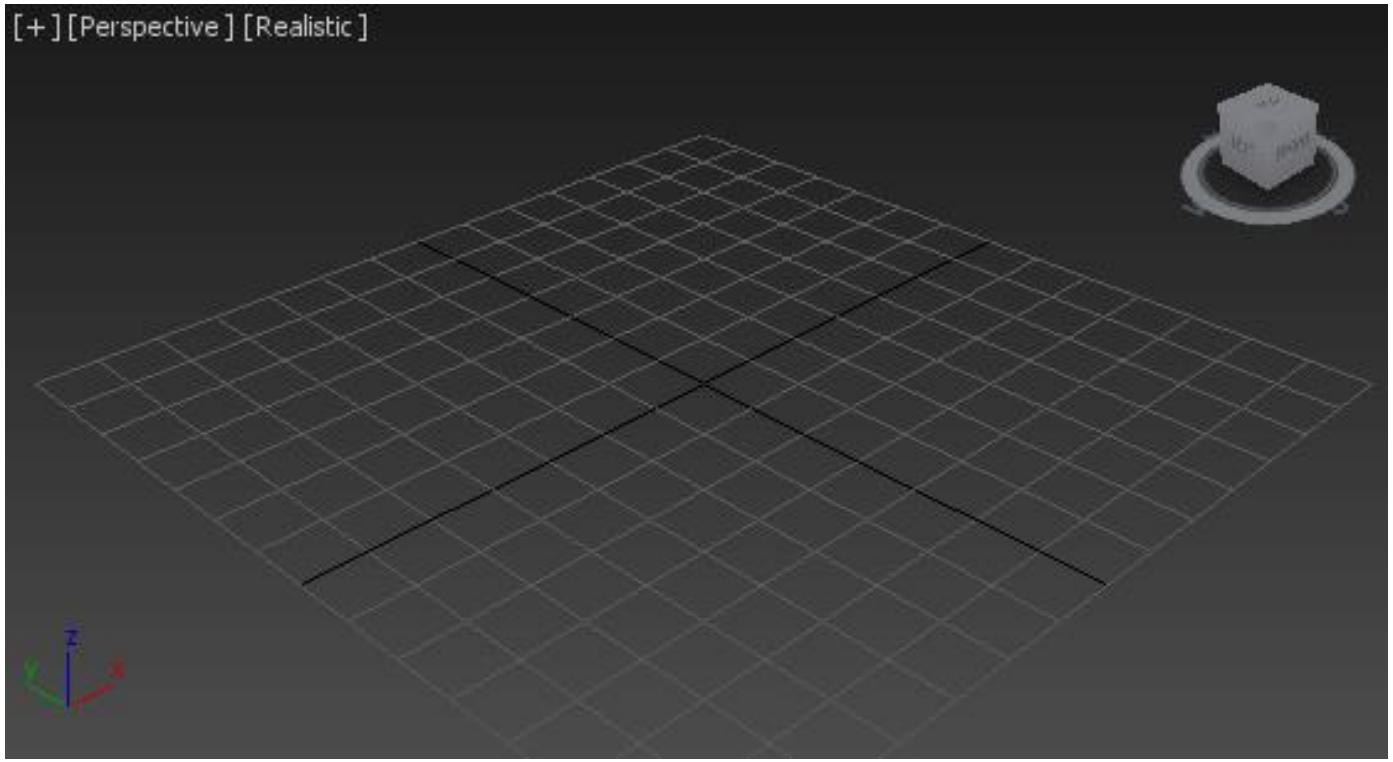
2. Posicionamiento

- Con respecto al software de tratamiento del posicionamiento, se realiza un proceso con una parte cíclica, que se corresponde con el ciclo de búsqueda que realiza el usuario.
- En este proceso podemos distinguir tres etapas:
  - Inicialización
  - ciclo de seguimiento
  - postproceso



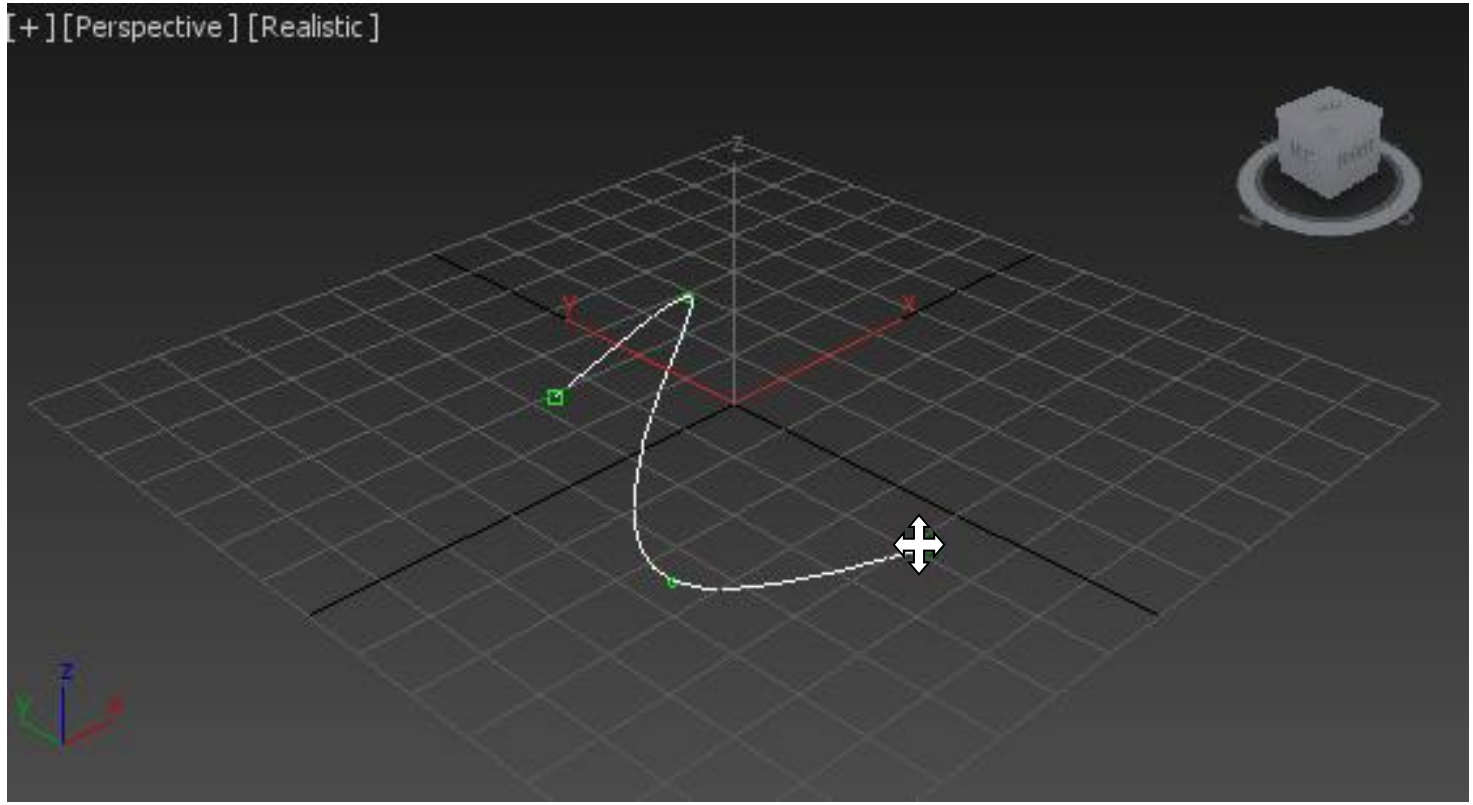
2. Posicionamiento

- El objetivo de las operaciones de posicionamiento es permitir la introducción de un punto.
- En la inicialización se muestra el estado inicial del dispositivo. La rejilla es frecuentemente usada como técnica.



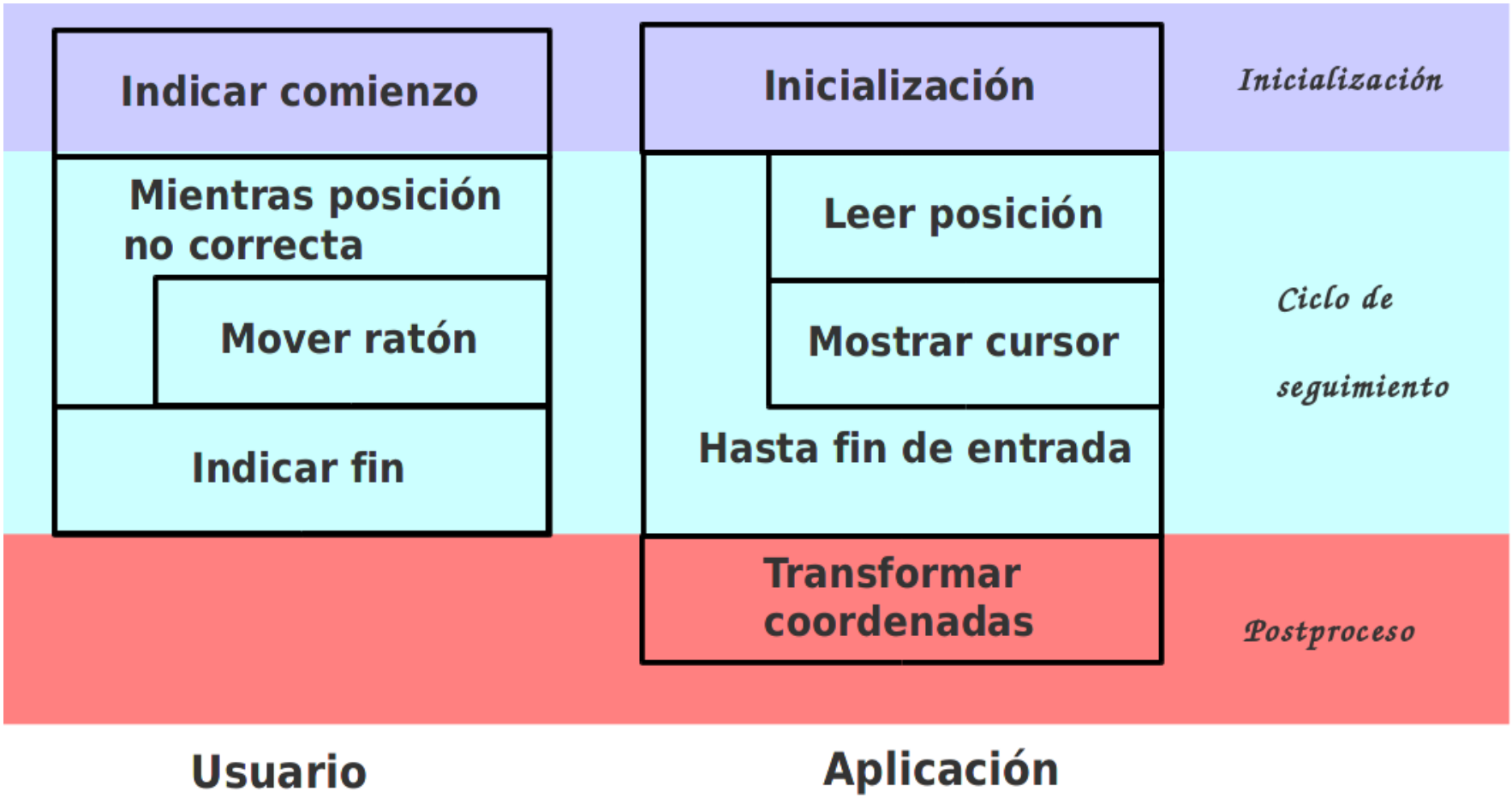
2. Posicionamiento

- En el seguimiento se usa con frecuencia la línea elástica como técnica.



2. Posicionamiento

- Correspondencia entre operaciones del usuario y del software de tratamiento

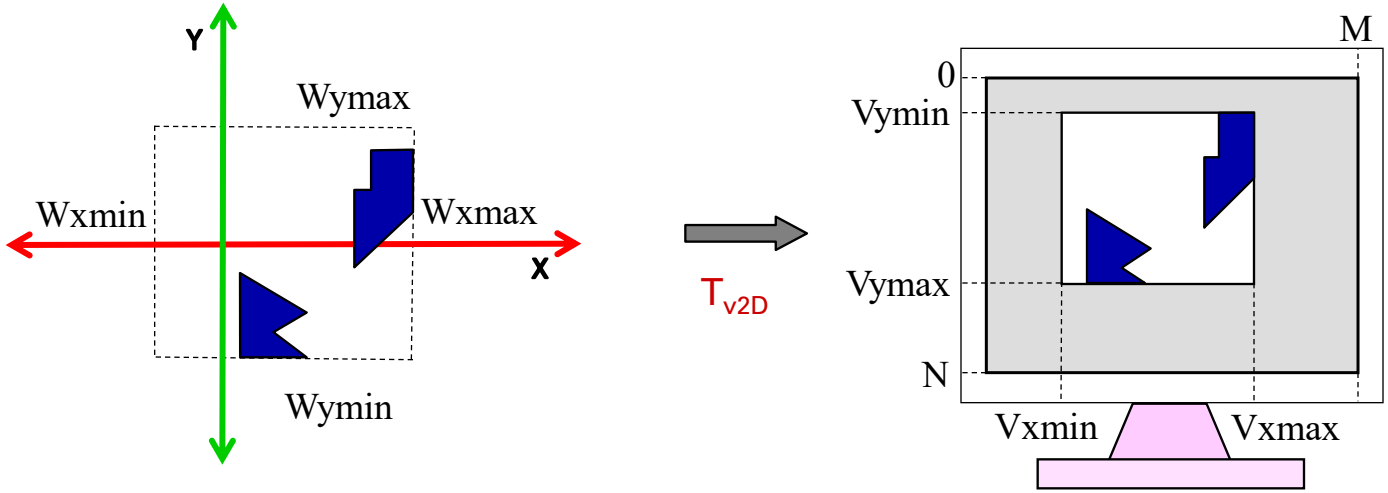


2. Posicionamiento

- La posición obtenida en la lectura está dada en coordenadas de dispositivo, por tanto, es necesario transformarla a coordenadas del mundo. Para ello se puede:
  - Invertir la transformación de visualización (si el modelo es 2D)
  - Invertir la transformación de visualización (si el dispositivo de entrada es 3D)
  - Utilizar vistas con tres proyecciones paralelas
  - Restringir el posicionamiento a un plano (calculando la posición sobre el plano)
  - Utilizar un cursor 3D

2. Posicionamiento (posicionamiento 2D)

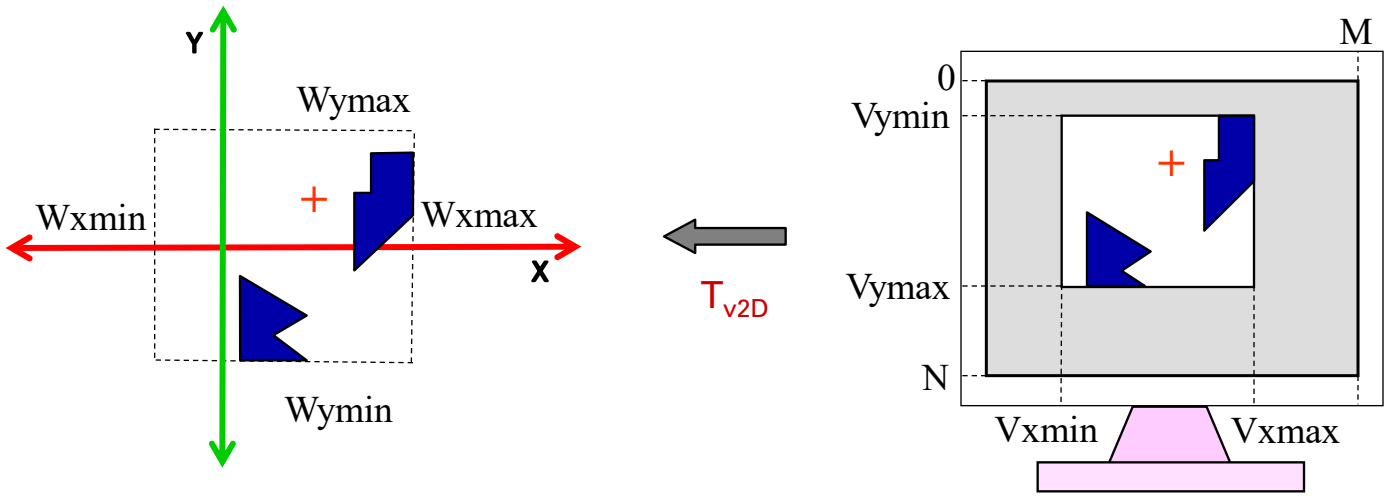
- Invirtiendo la transformación de visualización 2D
- Suponemos un viewport en el dispositivo no normalizado



$$T_{v2D} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -W_{xmin} & -W_{ymin} & 1 \end{pmatrix} \begin{pmatrix} \frac{Vxmax - Vxmin}{Wxmax - Wxmin} & 0 & 0 \\ 0 & \frac{Vymax - Vymin}{Wymax - Wymin} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ V_{xmin} & V_{ymin} & 1 \end{pmatrix} \begin{pmatrix} M & 0 & 0 \\ 0 & -N & 0 \\ 0 & N & 1 \end{pmatrix}$$

2. Posicionamiento (posicionamiento 2D)

- Invirtiendo la transformación de visualización 2D



$$\begin{pmatrix} 1/M & 0 & 0 \\ 0 & -1/N & 0 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -V_{xmin} & -V_{ymin} & 1 \end{pmatrix} \begin{pmatrix} \frac{Wxmax - Wxmin}{Vxmax - Vxmin} & 0 & 0 \\ 0 & \frac{Wymax - Wymin}{Vymax - Vymin} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ W_{xmin} & W_{ymin} & 1 \end{pmatrix}$$

2. Posicionamiento (posicionamiento 2D)

- Invirtiendo la transformación de visualización 2D

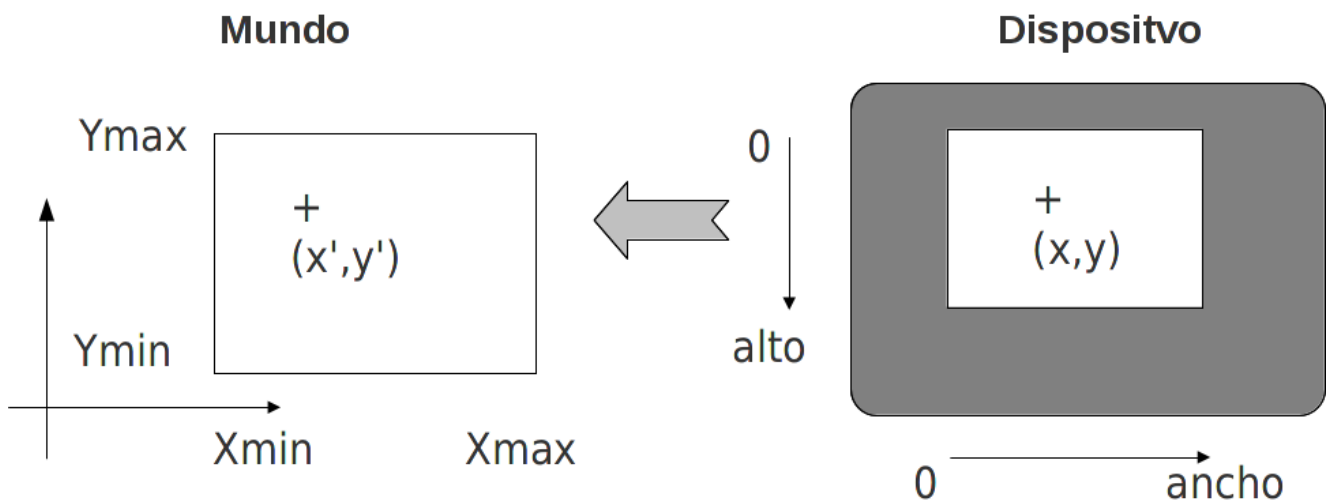
$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & N & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -V_{xmin} & -V_{ymin} & 1 \end{pmatrix} \begin{pmatrix} \frac{Wxmax - Wxmin}{Vxmax - Vxmin} & 0 & 0 \\ 0 & \frac{Wymax - Wymin}{Vymax - Vymin} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ W_{xmin} & W_{ymin} & 1 \end{pmatrix} =$$

$$\begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ -S_x V_{xmin} + W_{xmin} & NS_y - S_y V_{ymin} + W_{ymin} & 1 \end{pmatrix}$$

$$Sx = \frac{Wxmax - Wxmin}{Vxmax - Vxmin} \qquad Sy = \frac{Wymax - Wymin}{Vymax - Vymin}$$

2. Posicionamiento (posicionamiento 2D)

- Cuando el viewport es todo el dispositivo (0,0,ancho,alto), la expresión anterior se simplifica a:

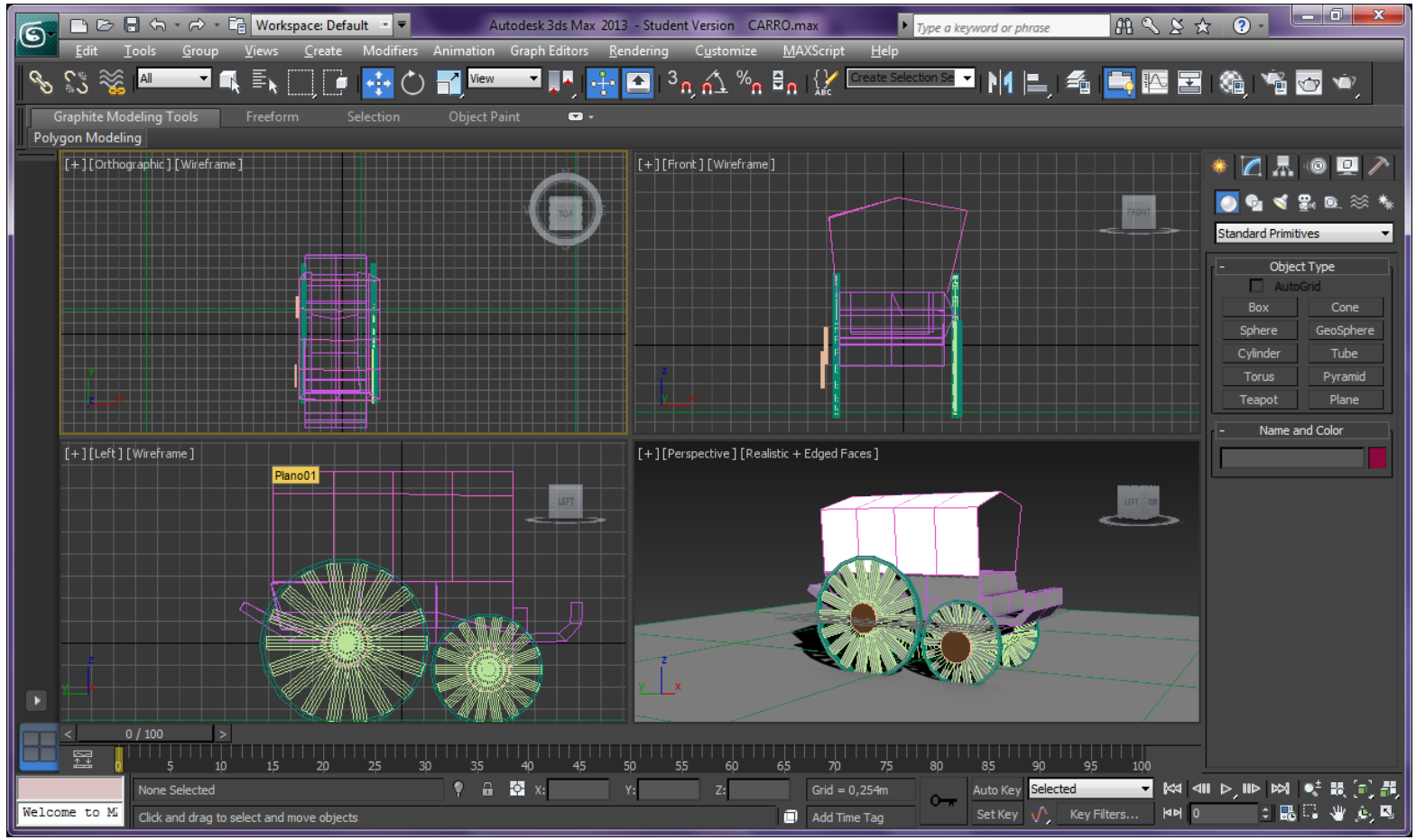


$$´x´ = x * \frac{Wxmax - Wxmin}{ancho} + Wx\,min$$

$$y´ = -y * \frac{Wymax - Wymin}{alto} + Wy\,max$$

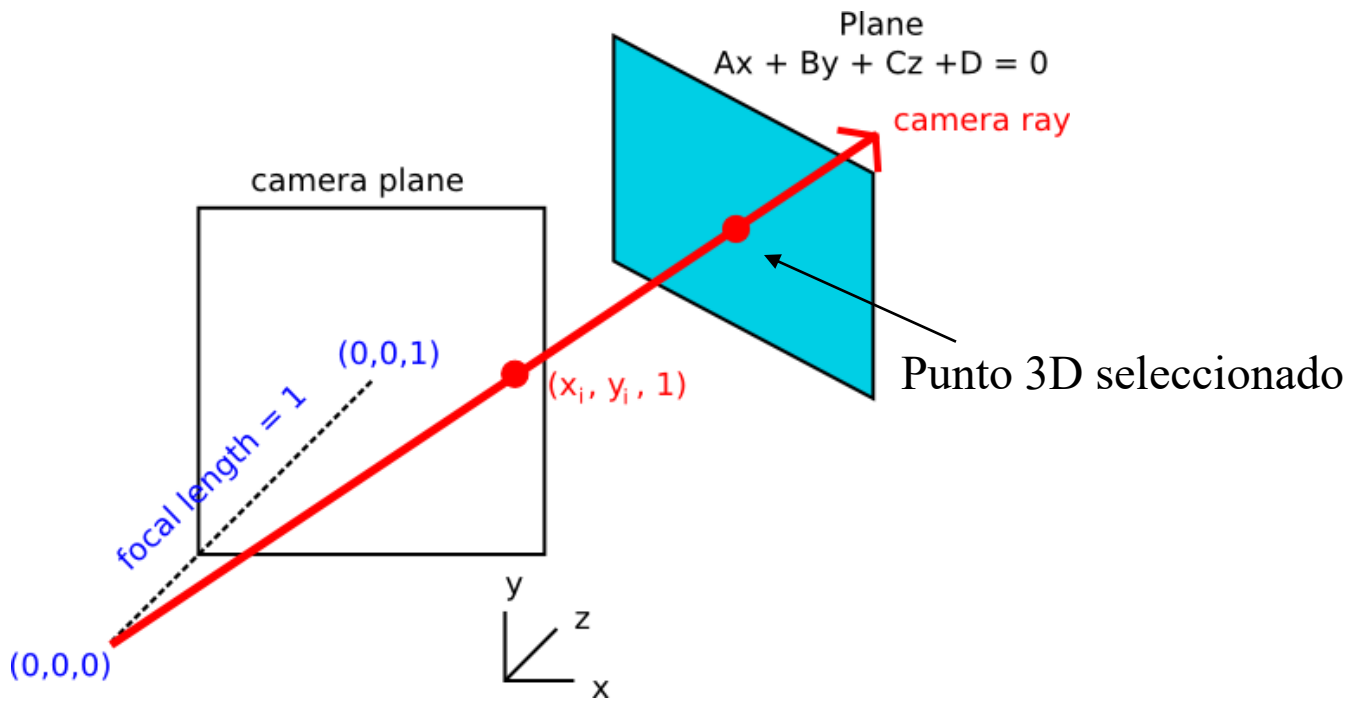
2. Posicionamiento (posicionamiento 3D)

- Vistas con tres posiciones paralelas.
  - En cada vista se fijan dos coordenadas



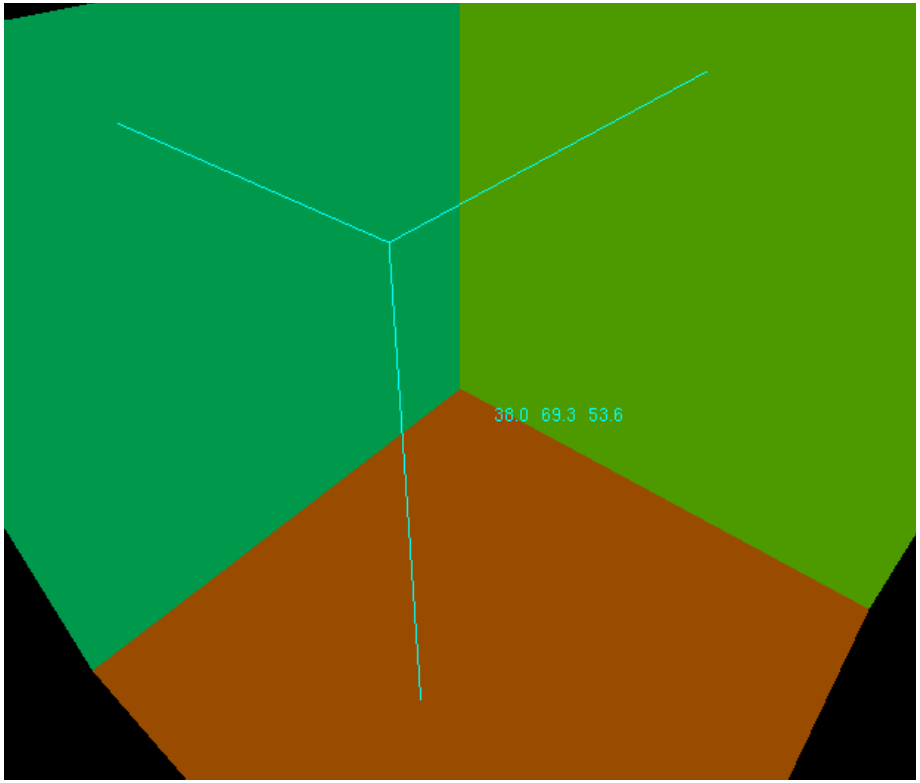
2. Posicionamiento (posicionamiento 3D)

- Restringir el posicionamiento a un plano.
  - Si se restringe la posición a un plano que no sea perpendicular al plano de proyección se puede obtener una posición 3D en dicho plano por intersección de la recta que pasa por el punto introducido (en el plano de proyección) y el centro de proyección.



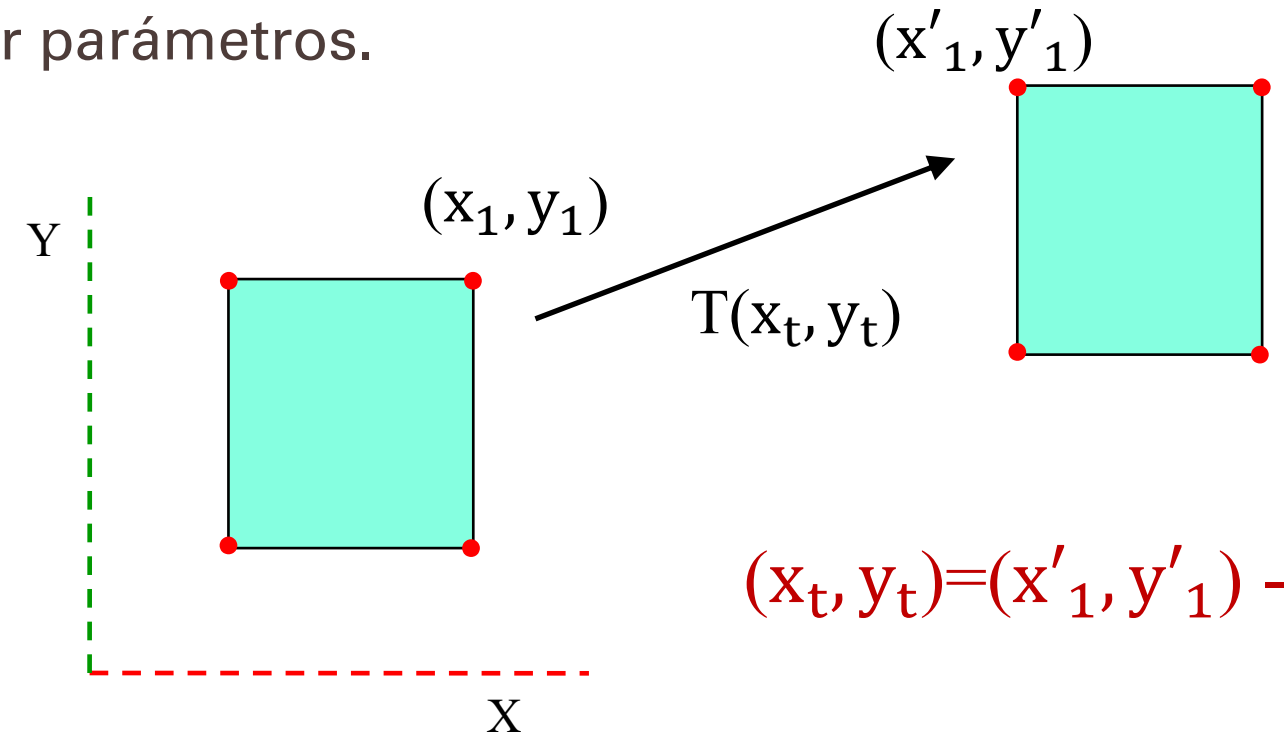
2. Posicionamiento (posicionamiento 3D)

- Cursor 3D.
  - La realimentación es un cursor 3D
  - El desplazamiento se realiza en el plano X-Z o en el X-Y en función de los botones del ratón que estén pulsados.



2. Posicionamiento (transformaciones geométricas)

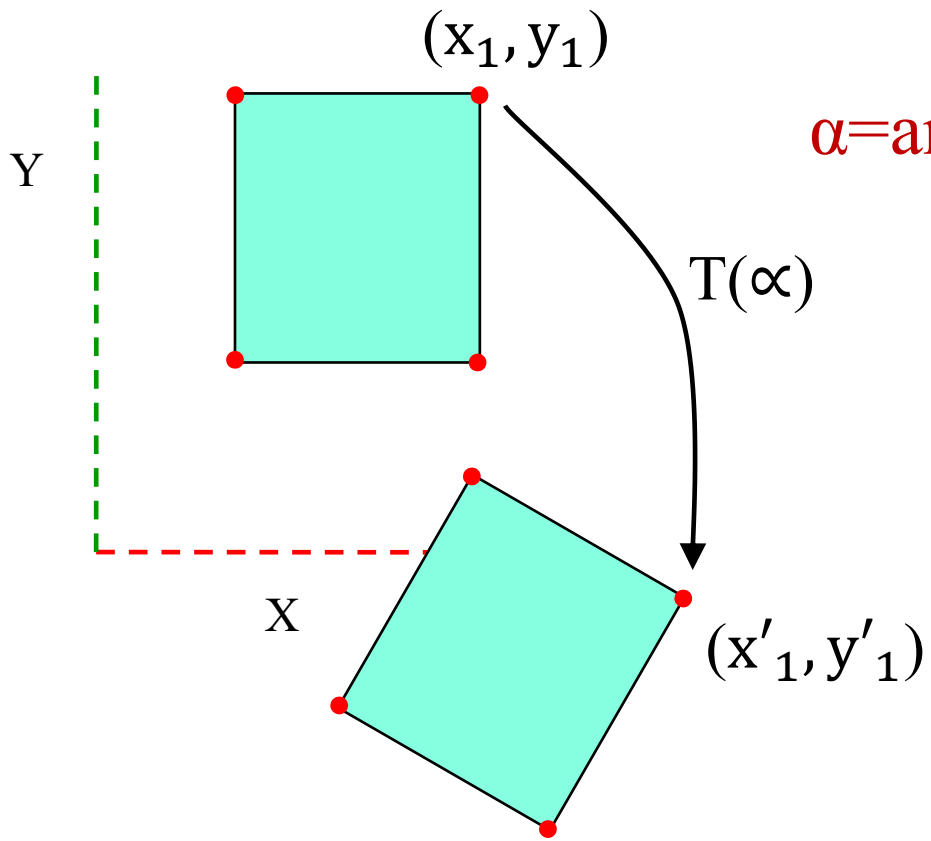
- La transformación se puede indicar con dos posicionamientos o solo uno, si se asume un punto de referencia (p.e. el centro del objeto).
- Traslación
  - Interacción: arrastre.
  - Calcular parámetros.



$$(x_t, y_t)=(x'_1, y'_1) - (x_1, y_1)$$

2. Posicionamiento (transformaciones geométricas)

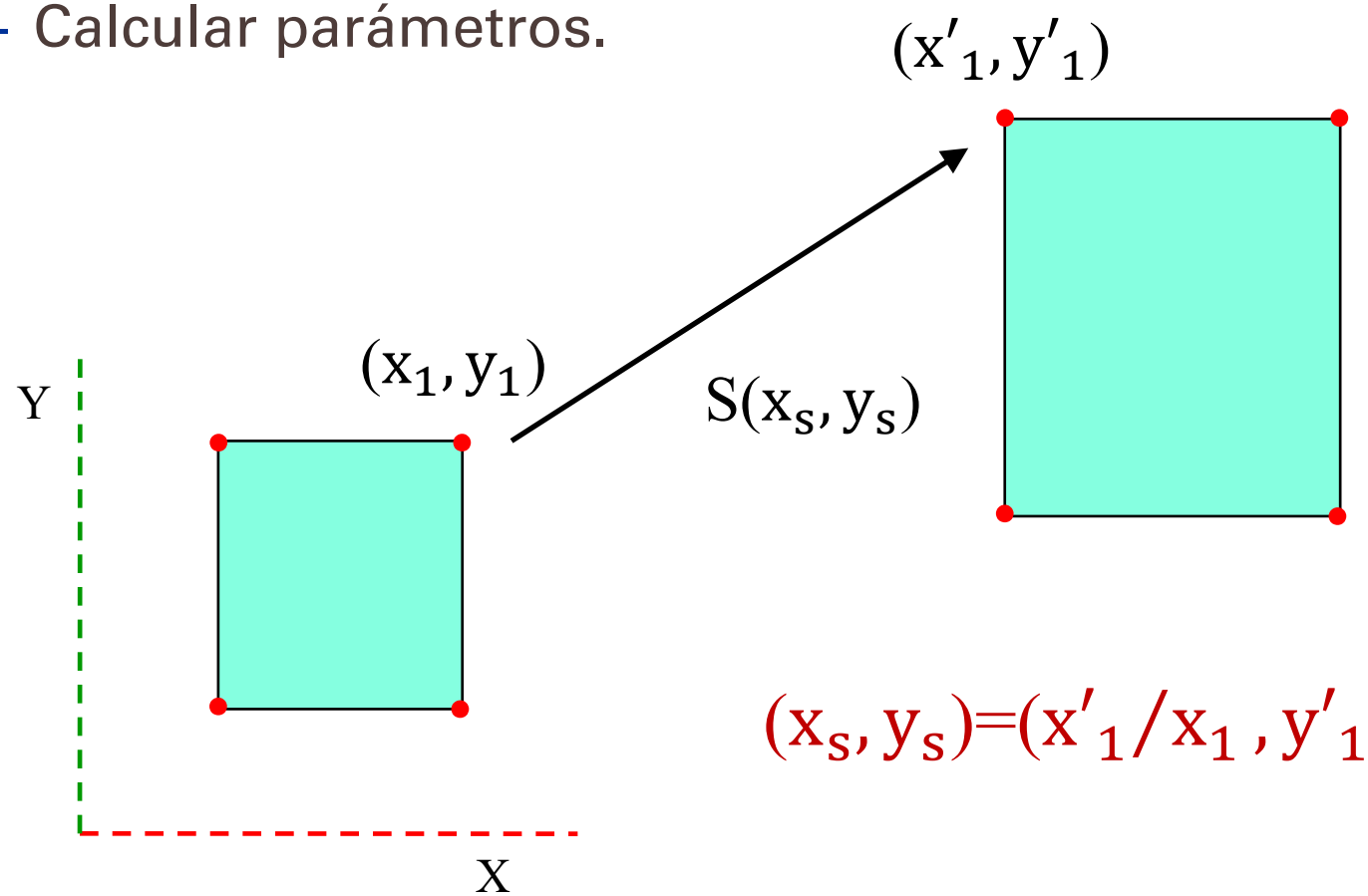
- Rotación
  - Interacción: arrastre.
  - Calcular parámetros.



$$\alpha = \text{angulo}(x'_1, y'_1) - \text{angulo}(x_1, y_1)$$

2. Posicionamiento (transformaciones geométricas)

- Escalado
  - Interacción: arrastre.
  - Calcular parámetros.



$$(x_s, y_s) = (x'_1/x_1 , y'_1/y_1)$$

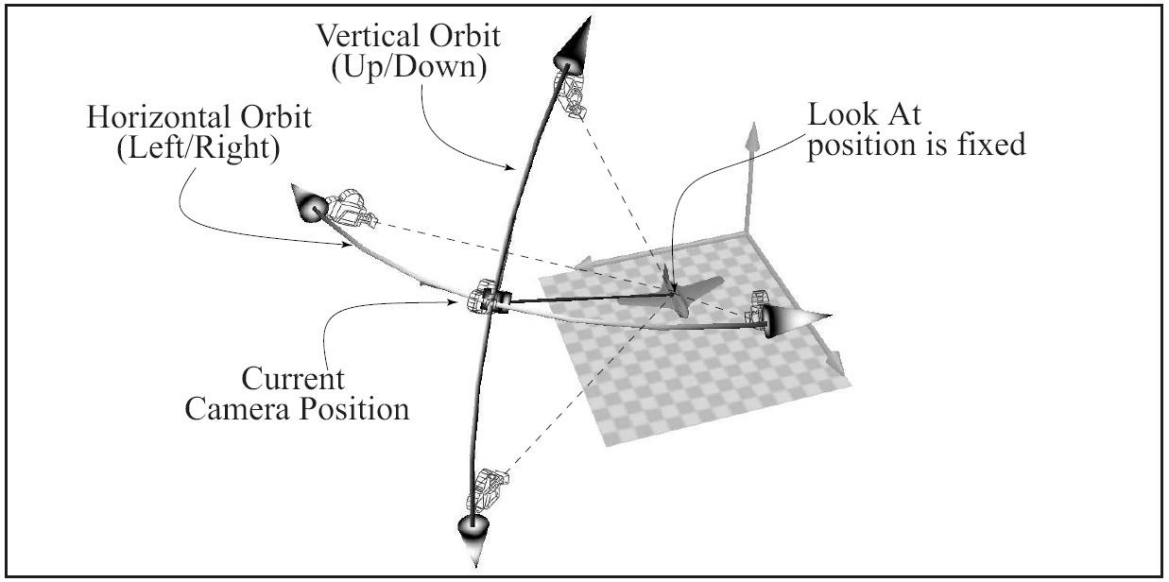
2. Posicionamiento (control de cámara)

- La cámara tiene demasiados parámetros para controlarlos todos interactivamente usando un ratón.

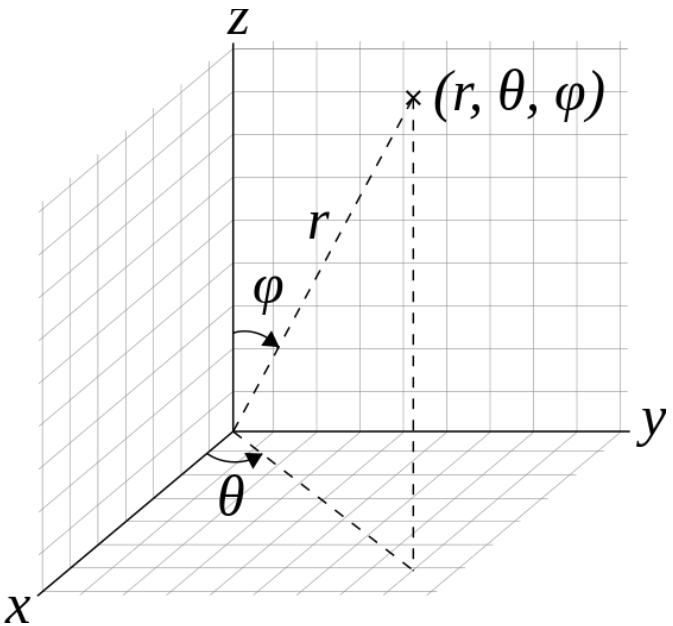
- Habitualmente se controla de forma interactiva un número reducido de parámetros, que se seleccionan dependiendo del tipo de aplicación:
  - visualización de modelos: posición (cámara orbital)
  - exploración de escenarios: dirección (cámara en primera persona)

2. Posicionamiento (cámara orbital)

- La cámara mira al objeto, moviéndose en torno a él sobre una esfera.
- La posición de la cámara se puede especificar en coordenadas esféricas.
- El vector de inclinación se debe calcular de forma que no sea perpendicular al plano proyección.



(K. Sung, P. Shirley, S. Baer: Essentials of Interactive Computer Graphics: Concepts and Implementation)



2. Posicionamiento (cámara orbital)

- Ejemplo de cámara orbital 1 (usando coordenadas esféricas)

```
extends Node3D

@export var latitud=0
@export var longitud=0
@export var radio=5.0  # posición inicial en z de la cámara
@export var incremento := 1  #velocidad angular en grado
@export var incremento2 := 0.05  # velocidad para trasladar la cámara

var acciones := ["orbitar_izquierda", "orbitar_derecha",
                 "orbitar_abajo", "orbitar_arriba", "alejar", "acercar"]

var cartesianas=[0,0,0]
var posicion=Vector3(0,0,radio)
var up = Vector3.UP

func _ready():
    var delta=0
    _process(delta)
```

2. Posicionamiento (cámara orbital)

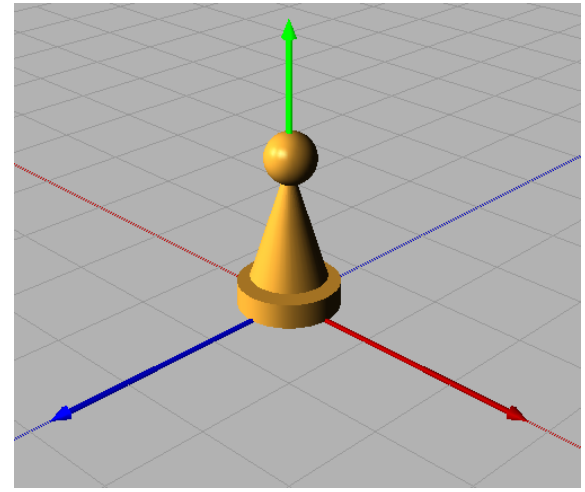
- Ejemplo de cámara orbital 1 (usando coordenadas esféricas)

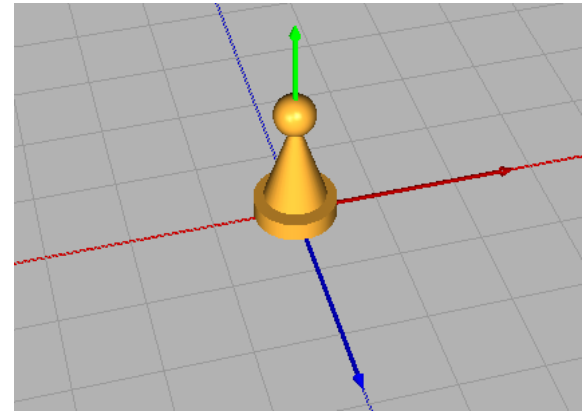
```
extends Node3D
...
func _process(delta):
    for action in acciones:
        if Input.is_action_pressed(action):
            match action:
                "orbitar_izquierda": longitud -= incremento
                "orbitar_derecha":   longitud += incremento
                "orbitar_abajo":  latitud -= incremento
                "orbitar_arriba": latitud += incremento
                "alejar":  radio += incremento2
                "acercar": radio -= incremento2
            esfe_cart(radio, latitud, longitud, cartesianas)    # cambio de coordenadas
            posicion=Vector3(cartesianas[0],cartesianas[1],cartesianas[2])

    $Camera3D.position = posición                # cámara definida por punto del observador,
    $Camera3D.look_at(Vector3(0, 0, 0),up)  #  punto de mira y vector inclinación
```

2. Posicionamiento (cámara orbital)

- Ejemplo de cámara orbital 1 (usando coordenadas esféricas)

```
func esfe_cart(radio: float , latitud: int , longitud: int,
                    cartesianas: Array ):
    var phi
    var theta
    latitud=latitud%360
    longitud=longitud%360

    phi=latitud*PI/180.0
    theta=longitud*PI/180.0
    cartesianas[0]=radio*sin(theta)*cos(phi)
    cartesianas[1]=radio*sin(phi)
    cartesianas[2]=radio*cos(phi)*cos(theta)

    if ((latitud > 90 && latitud <270) ||
       (latitud < -90 && latitud > -270)):
                    up[1] = -1
        else:
                    up[1] = 1
```





2. Posicionamiento (cámara orbital)

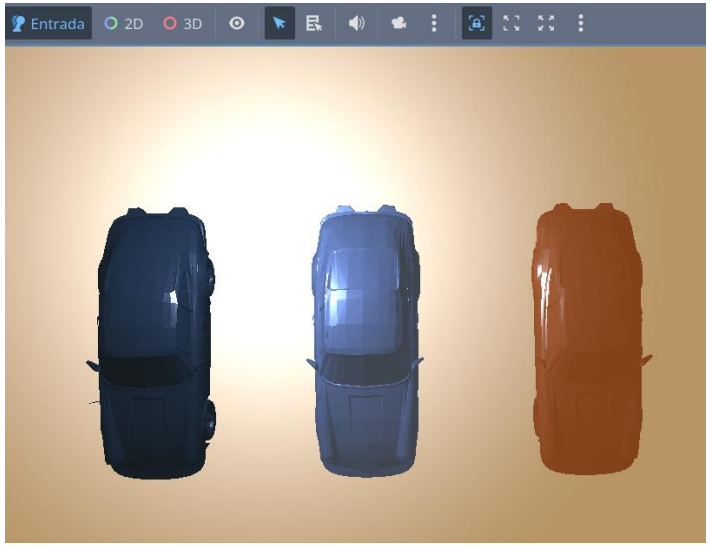
- Ejemplo de cámara orbital 2

```
# Valores iniciales para la cámara
extends Camera3D
@export var zoom_speed := 0.2
@export var rotate_speed := 0.01
@export var umbral := 0.05
var rot_x := 0.0
var rot_y := 0.0
var pos:=position.z
....
func _unhandled_input(event):
    if (event is InputEventMouseMotion and
        Input.is_mouse_button_pressed(MOUSE_BUTTON_RIGHT)):
        rot_y -= event.relative.x * rotate_speed
        rot_x -= event.relative.y * rotate_speed
        rot_x = clamp(rot_x, -PI/2, PI/2)  # evita la inversión al llegar a los polos
        position=Vector3(0.0,0.0,0)
        rotation = Vector3(rot_x, rot_y, 0)
        translate(Vector3(0,0,pos))
    ...
```

2. Posicionamiento (cámara orbital)

- Ejemplo de cámara orbital 2

```
func _unhandled_input(event):
    ... # continuación para el zoom
    if event is InputEventMouseButton:
        if event.button_index == MOUSE_BUTTON_WHEEL_UP:
            translate(Vector3(0, 0, -zoom_speed))
            pos-=zoom_speed
            if pos<umbral: pos=umbral
        elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
            translate(Vector3(0, 0, zoom_speed))
            pos+=zoom_speed
```



2. Posicionamiento (cámara en primera persona)

- La cámara está en un avatar o un personaje que se mueve en el escenario (punto de observación).

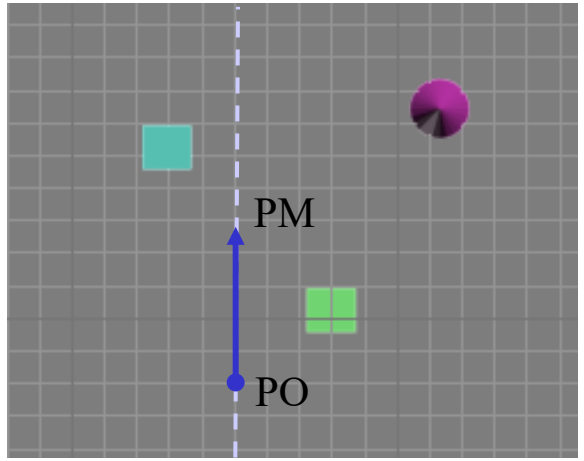
- El usuario puede controlar la dirección en la que mira la cámara (punto de mira), que se puede especificar como dos giros en direcciones ortogonales a la dirección de avance del personaje.

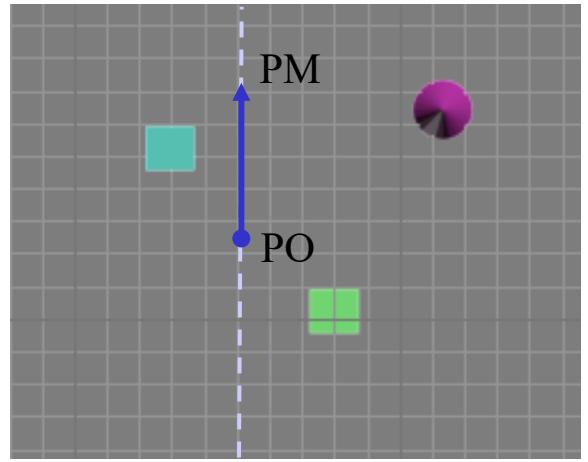
- El vector de inclinación se debe calcular de forma que no sea perpendicular al plano de proyección.

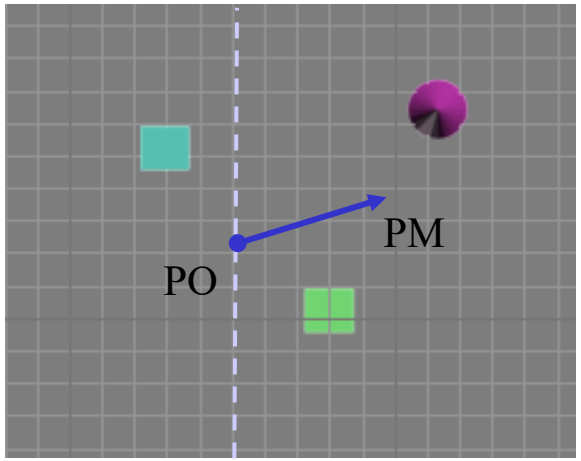
- Una cámara en tercera persona puede seguir al personaje.

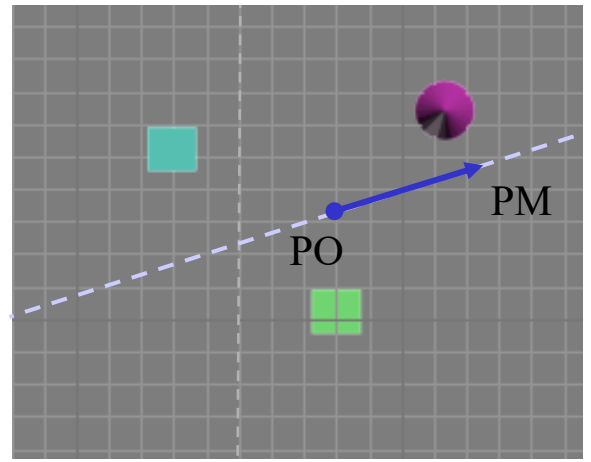
2. Posicionamiento (cámara en primera persona)

- Ejemplo cámara simple en primera persona (solo giro en Y).









2. Posicionamiento (cámara en primera persona)

- Ejemplo cámara simple en primera persona (solo giro en Y).

```
extends Node3D

@export var PM:=[0.0, 0.0, 1.0]  # punto de mira inicial
@export var PO:=[0.0, 0.0, 4.0]   # punto del observador inicial
@export var rotar := 0.02    # velocidad angular en grado
@export var mover := 0.02 # velocidad de traslación

var acciones := ["rotar_izquierda", "rotar_derecha", "avanzar", "retroceder"]

var posicion:=Vector3(0,0,5.0)
var up = Vector3.UP

func _ready():
	var delta=0
	_process(delta)
```

2. Posicionamiento (cámara en primera persona)

- Ejemplo cámara simple en primera persona (solo giro en Y).

```
...
func _process(delta):
    for action in acciones:
        if Input.is_action_pressed(action):
            match action:
                "rotar_izquierda": rotar_camara(PO, PM, -rotar)
                "rotar_derecha":   rotar_camara (PO, PM, rotar)
                "avanzar":      mover_camara(PO, PM, mover)
                "retroceder":  mover_camara(PO, PM, -mover)
            posicion=Vector3(PO[0], PO[1], PO[2])

    $Camera3D.position = posicion
    $Camera3D.look_at(Vector3(PM[0], PM[1], PM[2]), up)
```

2. Posicionamiento (cámara en primera persona)

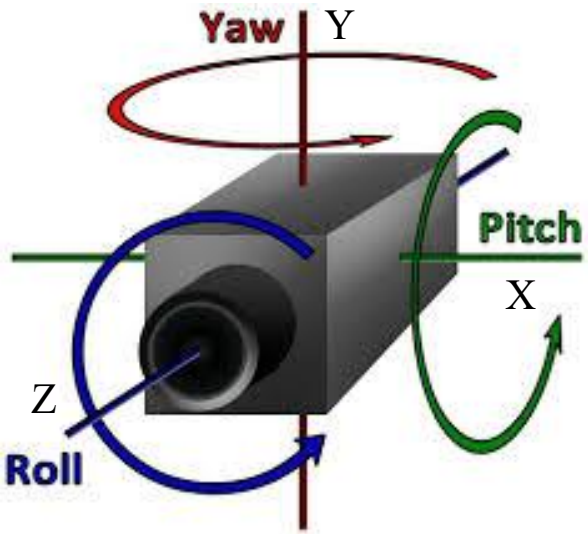
- Ejemplo cámara simple en primera persona (solo giro en Y).

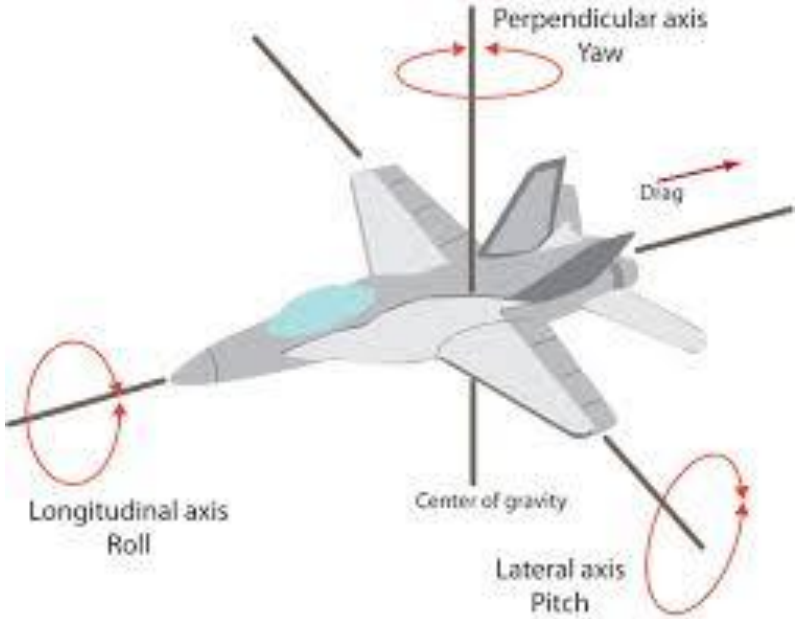
```
...
func mover_camara (PO: Array, PM: Array, t: float):
    var direccion:= Vector3(0.0, 0.0, 0.0)
    for i in range (0,3): direccion[i] = PM[i] - PO[i]

    PM[0]  = PM[0]  + direccion[0] * t
    PM[2]  = PM[2]  + direccion[2] * t
    PO[0]  = PO[0] + direccion[0] * t
    PO[2]  = PO[2] + direccion[2] * t

func rotar_camara (PO: Array, PM: Array, angulo: float):
    var direccion:= Vector3(0.0, 0.0, 0.0)
    for i in range (0,3): direccion[i] = PM[i] - PO[i]

    PM[0] = PO[0] + cos(angulo)*direccion[0] - sin(angulo)*direccion[2]
    PM[2] = PO[2] + sin(angulo)*direccion[0] + cos(angulo)*direccion[2]
```

2. Posicionamiento (cámara: ángulos de Euler)

- La orientación de la cámara (punto de mira de la cámara) se puede expresar como tres giros (uno respecto a cada eje).

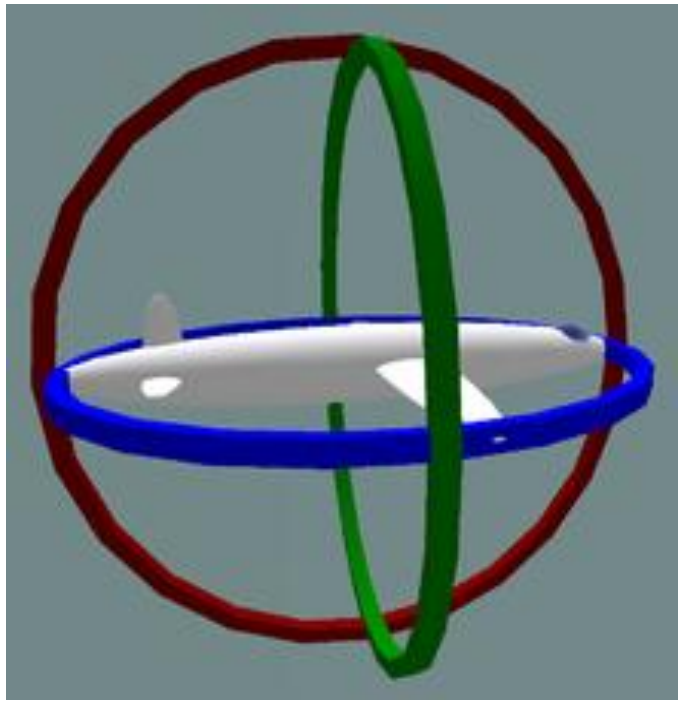
- Esta representación de la orientación se conoce como ángulos de Euler que permite controlar la orientación con tres grados de libertad (incluyendo el control sobre el vector de inclinación).

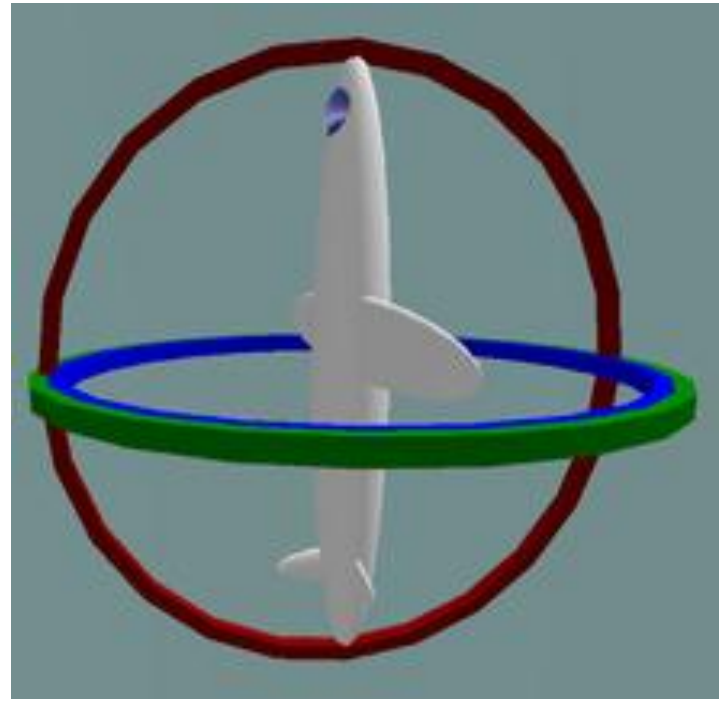




2. Posicionamiento (cámara: ángulos de Euler)

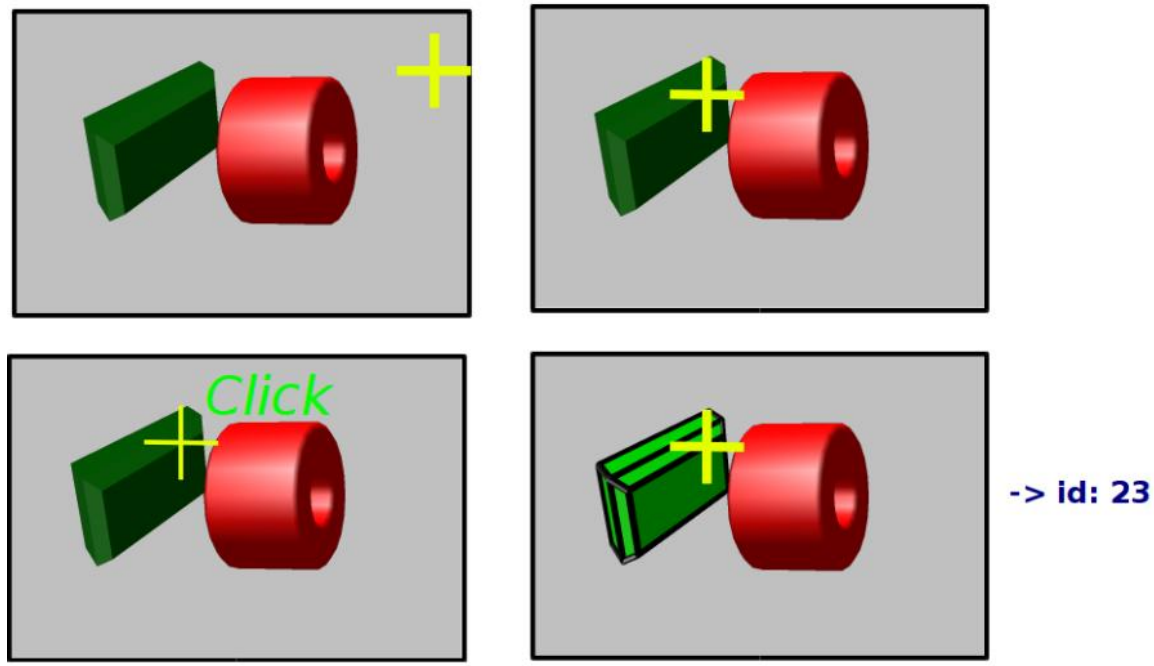
- La utilización de ángulos de Euler tiene el problema de generar configuraciones en las que dos ejes se alinean provocando la pérdida de uno de los grados de libertad.
- Este fenómeno se conoce como gimbal lock.





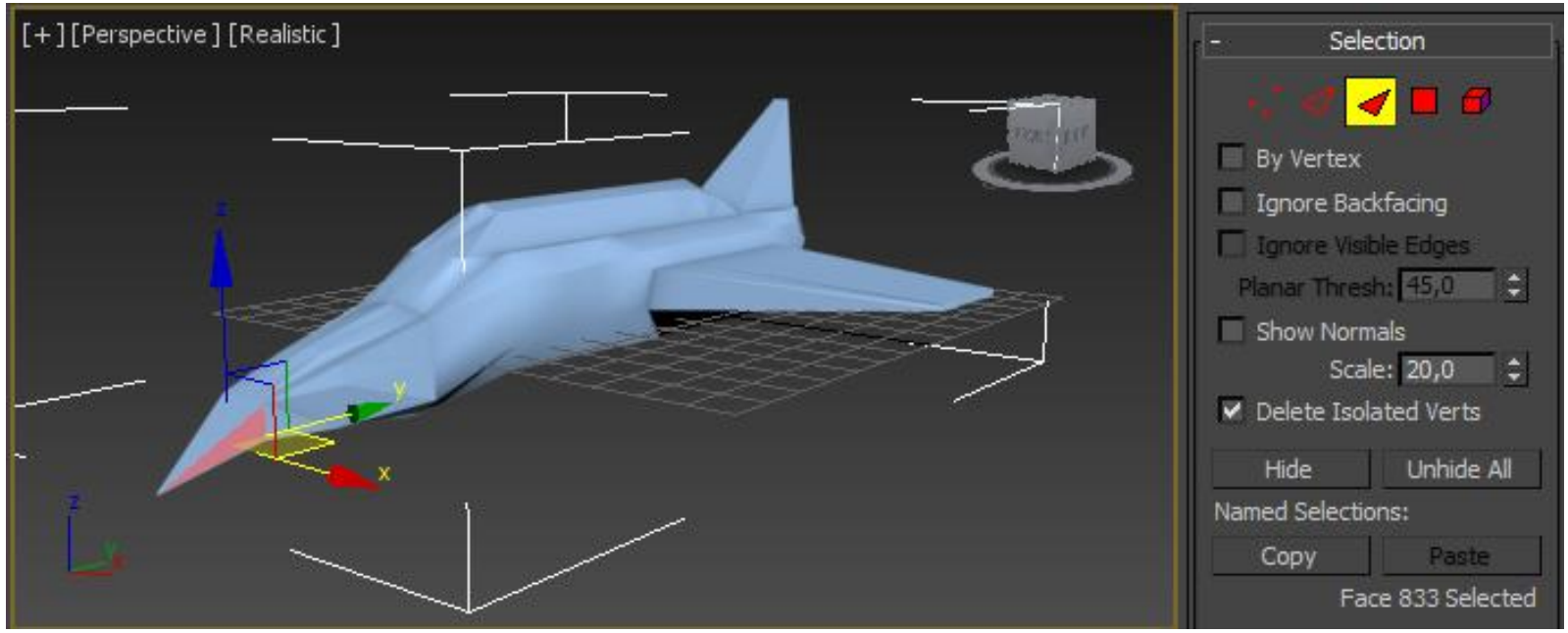
3. Selección

- La selección permite al usuario referenciar componentes de un modelo geométrico.
- Es esencial en cualquier aplicación gráfica que requiera la edición de un modelo.
- El proceso de selección suele realizarse como un posicionamiento y una búsqueda dentro del modelo geométrico.



3. Selección

- Para poder realizar la selección los componentes de la escena deben tener asociados identificadores o etiquetas.
- Éstas se pueden asociar a distinto nivel, permitiendo controlar el ámbito de la selección: vértice, primitiva, objeto

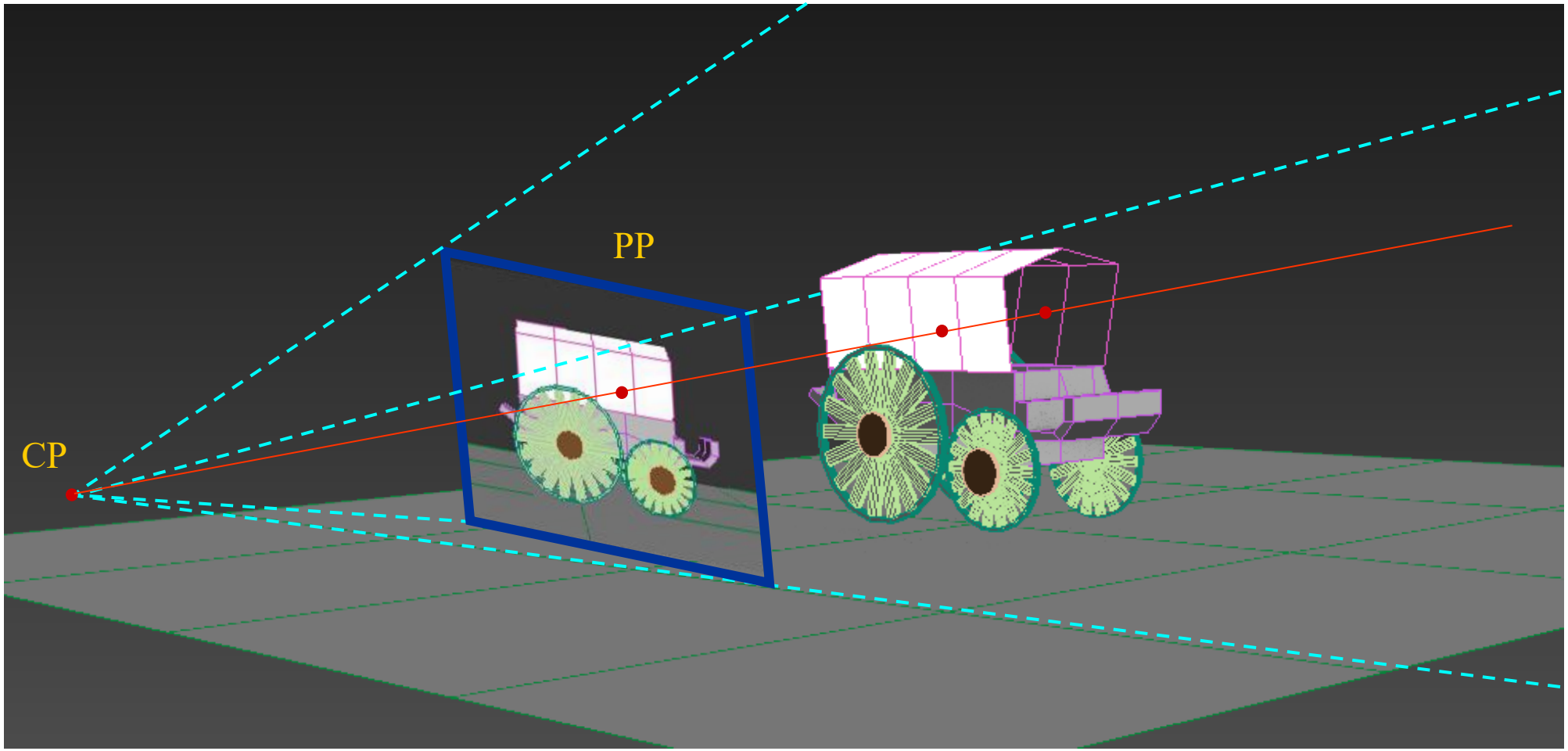


3. Selección (métodos)

- La asociación entre posiciones de pantalla y elementos del modelo se puede establecer de usando distintos métodos de selección:
  - Intersección rayo escena
  - Identificando los objetos incluidos en un subvolumen de visión centrado en la posición dada (OpenGL)
  - Codificando el identificador de objeto como color y leyendo del frame buffer

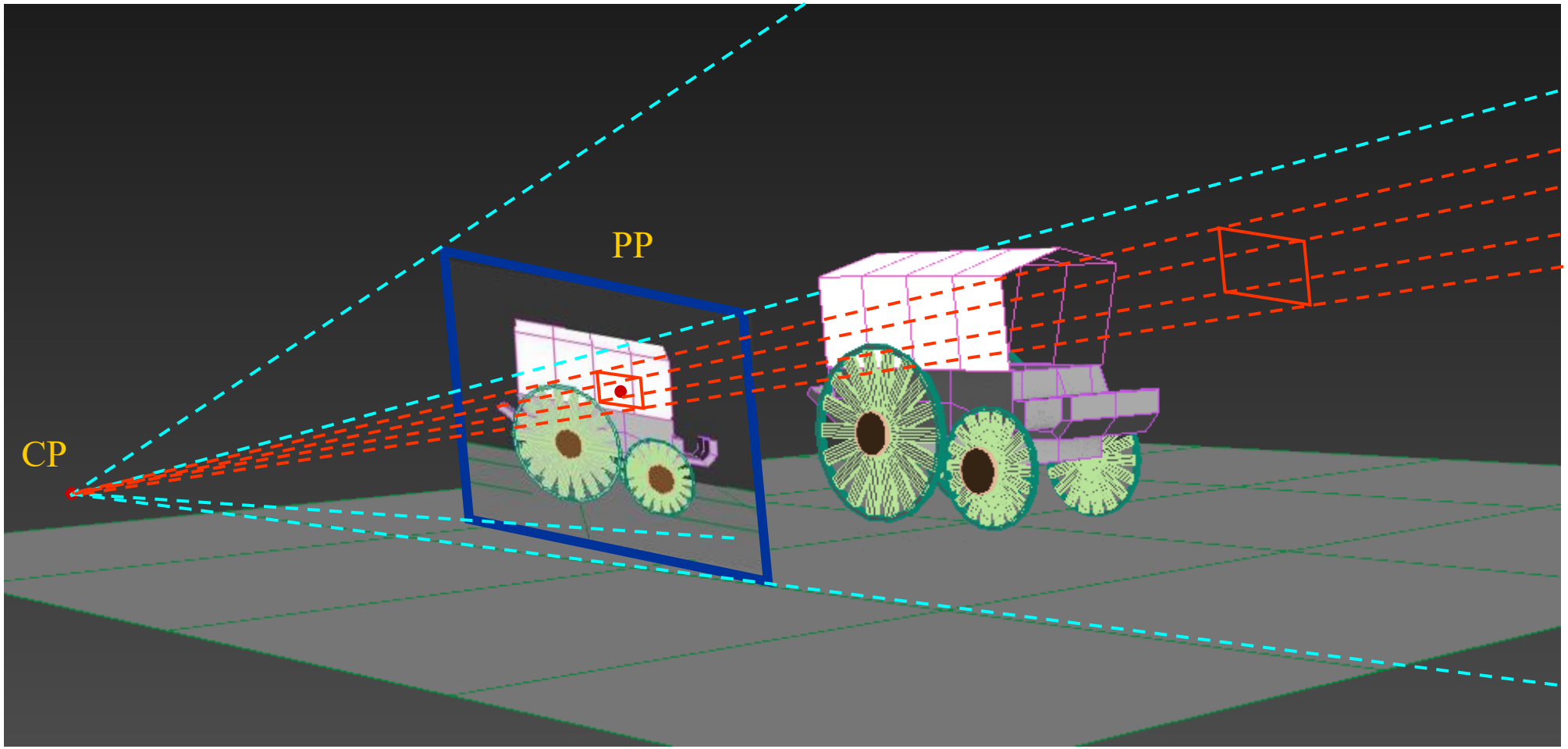
3. Selección (métodos)

- Intersección rayo escena



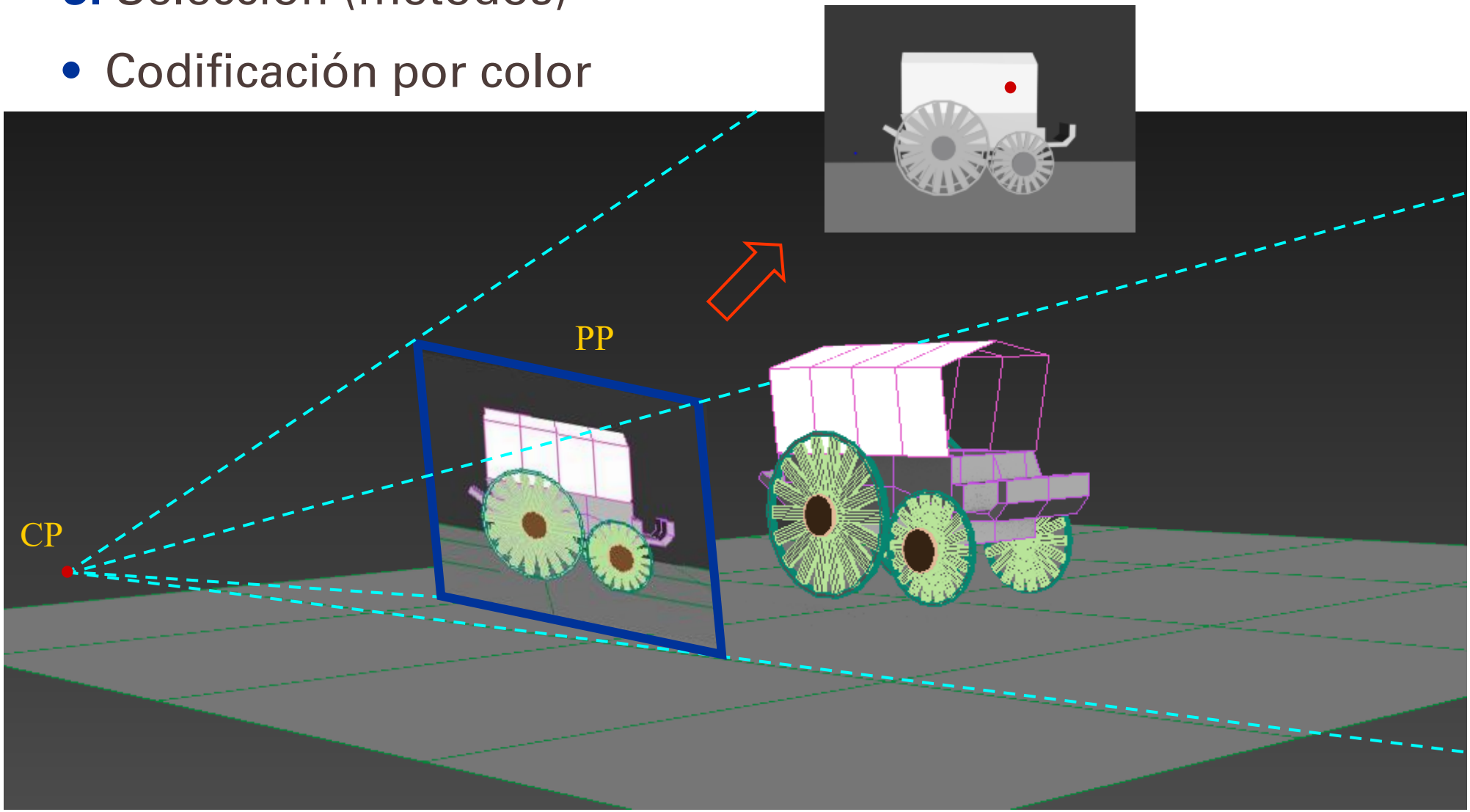
3. Selección (métodos)

- Subvolumen de visión



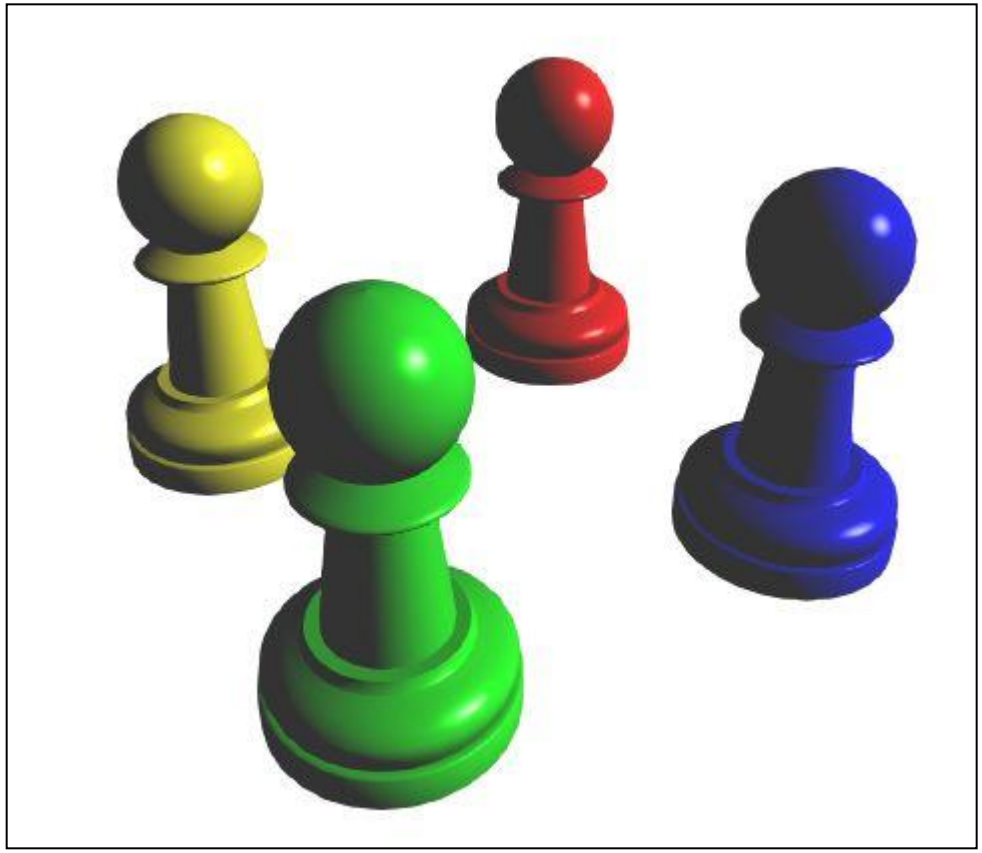
3. Selección (métodos)

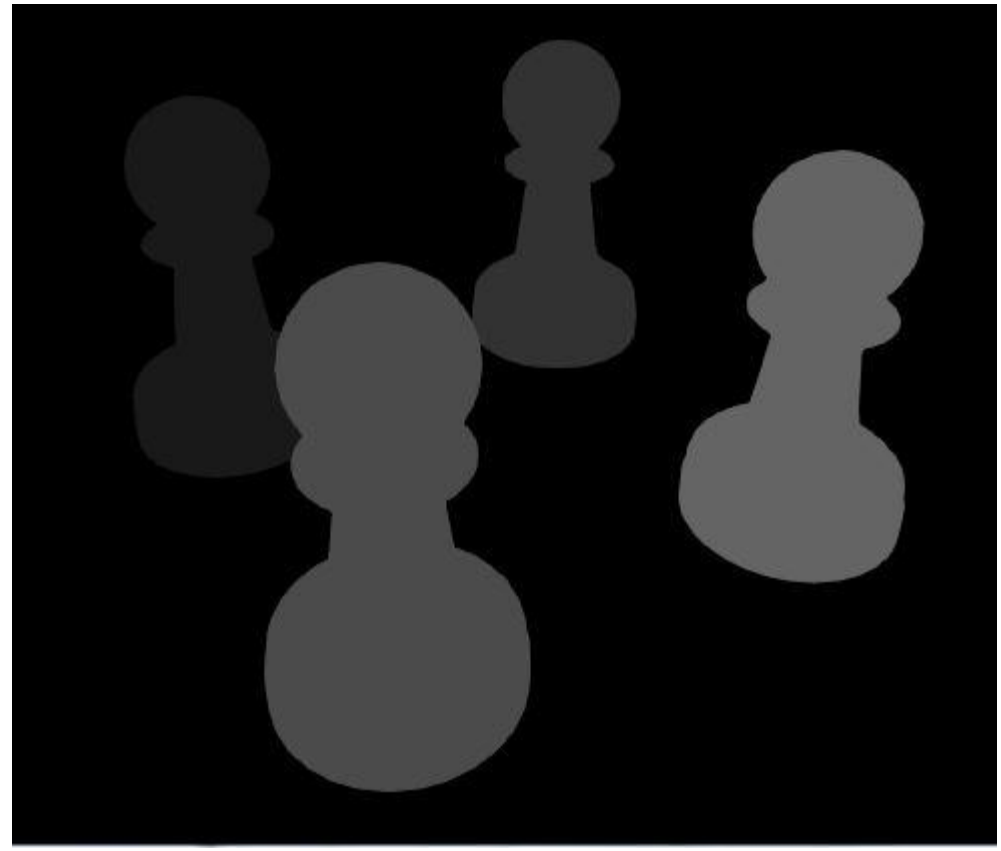
- Codificación por color



3. Selección (métodos)

- Codificación por color





4. Animación

- Animación: Creación de la ilusión de que las cosas cambia

- No sólo debemos atenernos al cambio de los objetos de sitio, sino también a los cambios de forma, los cambios de color o intensidad de la luz.

- La ilusión de la animación se debe al fenómeno de la persistencia de la visión. Cuando presentamos rápidamente una serie de imágenes sucesivas que varían ligeramente, el sistema visual las mezcla.

Percepción de movimiento = 24 imágenes por segundo

4. Animación

- Existen diversos criterios de clasificación de la animación:
  - Según el agente que produce la animación:

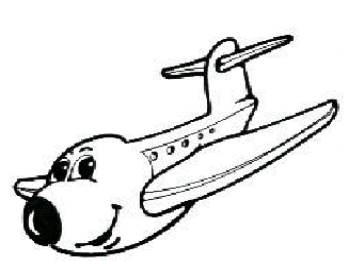
    Hombre → Animación convencional
    Ordenador → Animación por ordenador

  - Según el medio o herramienta que se use:

    Mano → Animación convencional
    Ordenador → Animación asistida por ordenador

  - Según la apariencia visual:

    Aspecto bidimensional
    Aspecto tridimensional



4. Animación

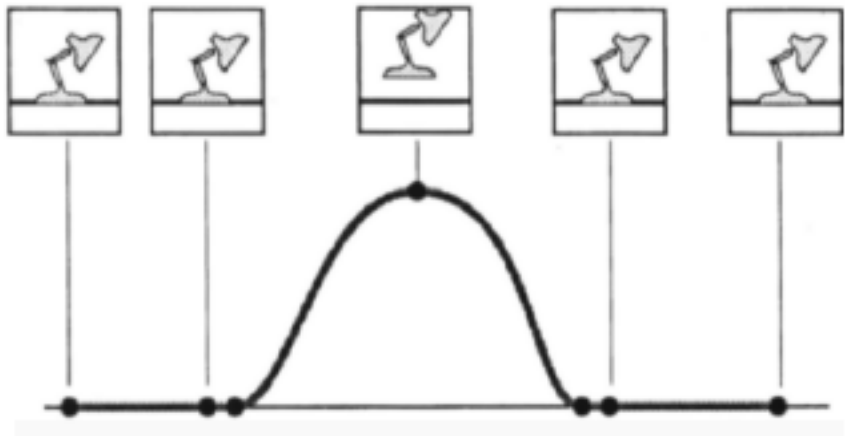
- La animación por ordenador está orientada sobre todo a animación tridimensional, manejando entornos más complejos que la tradicional.
- Ventaja: Automatismo y manejo de grandes cantidades de información.
- Desventaja: Una buena animación requiere igual tiempo que hecha a mano

4. Animación

- En la animación por ordenador, éste tiene un papel verdaderamente importante. No es una herramienta de apoyo, sino un elemento básico.
- La flexibilidad de la animación por ordenador reside en la posibilidad de crear mundos virtuales, con objetos y reglas virtuales que pueden ser cambiadas modificando la definición de los mismo.
- Centrándonos en la modificación de los objetos de una escena, podemos usar las siguientes técnicas, que pueden combinarse:
  - fotogramas clave (keyframe)
  - esqueleto
  - simulación física
  - procedural

4. Animación (keyframe)

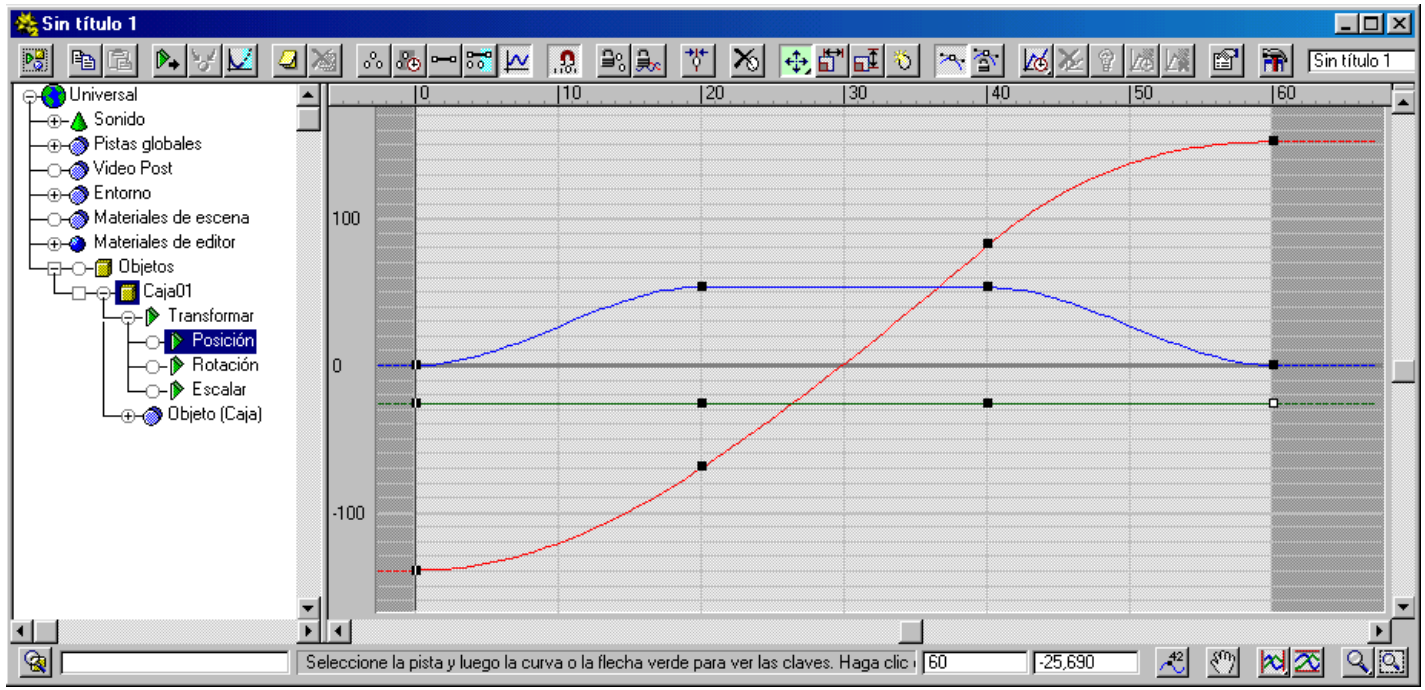
- Se basa en la definición de una serie de posiciones o estados clave en determinados instantes de tiempo para un objeto dado.
- Después, se generarán los pasos intermedios mediante el concepto de intercalado: dadas dos situaciones clave inicial y final, obtener las intermedias mediante interpolación entre ellas.
- Las posiciones clave han de ser aquellas de las que fácilmente se pueden obtener otras posiciones intermedias. No es fácil que el ordenador haga un buen intercalado si los fotograma clave son muy diferentes



4. Animación (keyframe)

- Se suelen utilizar curvas de movimiento que sirven para realizar diagnosis de la animación, así como para modificarla editando las curvas.

- Normalmente se incluyen tres curvas de movimiento para cada uno de los ejes: X,Y o Z (translación, rotación , escalado)



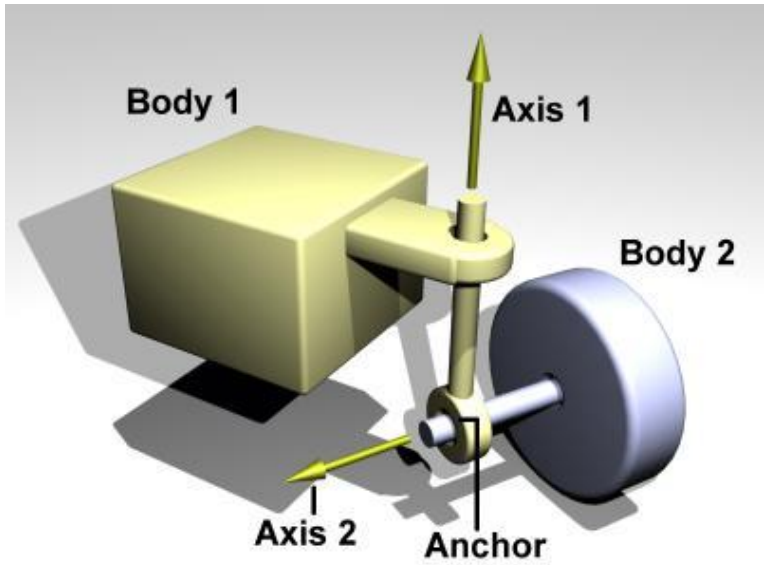
4. Animación (esqueleto o armature)

- Un esqueleto es un modelo simplificado del personaje, formado por segmentos rígidos (huesos o bones) unidos por articulaciones.

- En vez de utilizar la figura completa para realizar el intercalado, se utiliza el esqueleto, donde es más fácil hacer corresponder posiciones entre dos keyframes.

4. Animación (simulación física)

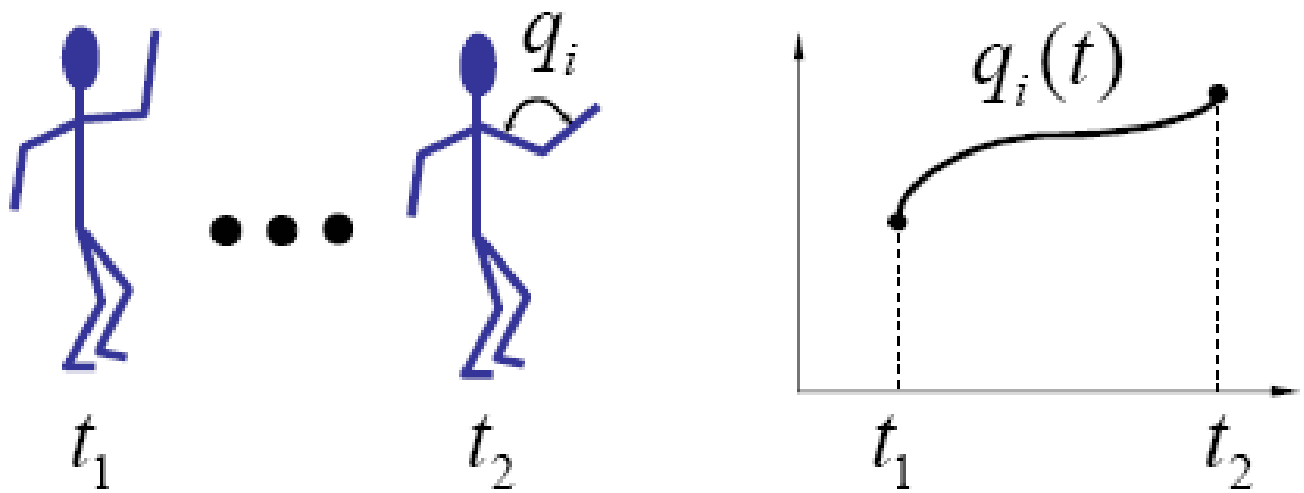
- Para escenas con modelos físicos simples se puede calcular la configuración del modelo en cada fotograma usando las leyes de la mecánica clásica.
- Librerías para simulación física:
  - ODE: Open Dynamic Engine.
  - Newton: Newton Physics Engine
  - Bullet: Physics Library



4. Animación (simulación física)

- Cinemática inversa
  - Se mueve las partes más bajas de la jerarquía de un objeto y el movimiento se trasmite a los ancestros de la mismas (igual que ocurre en la realidad).
  - Por ejemplo, si queremos acercar la mano de un personaje a un objeto, se colocan los dedos sobre el objeto, y el resto del brazo los seguirá automáticamente.
  - La cinemática inversa es desde el punto matemático un problema mal definido (hay diversas soluciones)
  - Debemos indicar el modo exacto en que las articulaciones se coordinan, y las limitaciones de sus movimientos (por ejemplo, valores de rotación) para limitar el número de soluciones.

4. Animación (procedural)

- El comportamiento del objeto se describe mediante procedimientos (funciones).

- Es útil cuando el comportamiento es fácil de generar pero difícil de simular físicamente.